

Project Report
On
Author's Haven Web-Application

Project work submitted in partial fulfilment of the requirement for the
award of the degree.

Bachelor of Computer Applications

By
Manha Fatima (107223861053)



**BHAVAN's VIVEKANANDA COLLEGE OF SCIENCE,
HUMANITIES & COMMERCE SAINIKPURI,
SECUNDERABAD – 500094**

Re-Accredited with 'A' Grade by NAAC Autonomous College – Affiliated to Osmania University

ACKNOWLEDGEMENT

Apart from the efforts of me, the success of any project depends largely on the encouragement and guidelines of many others. I take this opportunity to express my gratitude to the people who have been instrumental in the successful completion of the project.

Firstly, I would like to thank our Principal, **Dr. G.S.V.R.K. Choudary**, Bhavan's Vivekananda College, for having faith in me and believing that I am capable enough to go on with the project.

Secondly, I would like to thank **Ms. K. V. B. Saraswathi Devi**, Head of the Department of Computer Science, **Dr. G. Mahesh Kumar**, BCA Course Coordinator, and other faculty members for their help during my project completion in the college. I take it as a great privilege for having done the project for Bhavan's Vivekananda College Science, Humanities & Commerce, Sainikpuri.

Finally, I would like to convey my profound gratitude to my project guide **Mr. K. SRINIVASA RAO, Lecturer, Department of Computer Science** for giving me this project in the college and enabling me by his valuable guidance and support.

The guidance and support received from all the members who have contributed to this project, was vital for the success of the project. I am grateful for their constant support and help.

Manha Fatima
(107223861053)

DECLARATION

I hereby declare that the project work entitled “**Author’s Haven**” submitted to the Bhavan’s Vivekananda College, Sainikpuri, Secunderabad, is a record of an original work done by me under the guidance of **Mr. K. Srinivasa Rao**, Lecturer, Department of Computer Science, Bhavan’s Vivekananda College, Sainikpuri, Secunderabad.

I further certify that the work has not been submitted to any other Institution for the award of any degree or diploma.

Manha Fatima
(107223861053)

ABOUT THE ORGANIZATION

Bhavan's Vivekananda College of Science, Humanities and Commerce was in 1993 under the aegis of the Bharatiya Vidya Bhavan. Ever since its inspection, the college has been providing quality education to all students in large variety of courses. The college continues to provide all students a solid foundation for further educational opportunities and the skill for various career opportunities upon graduation.

The college offers a peaceful atmosphere, which is ideal for academic pursuits and for the overall development of the students. The college seeks to integrate the student's program of study and the development of skills, which includes critical thinking, problem solving, written and oral communication and encouraging research techniques in various fields. It seeks to learn, to adapt and to lead in the creation of a pool of committed and competent individuals indicated to the process of nation building.

The college has a dedicated set of staff, lecturers and support staff. The lecturers, apart from teaching, also imbibed in principles of discipline, commitment and hard work and to strive towards achieving the goals. They support the students in all possible ways and help students to solve their various problems and encourage the students to perform well in both academics and other co-curricular activities. Overall, the college, with its holistic atmosphere aided for learning, the principles and values that are imbibed into the students, the excellent staff headed by an able and strong Principle dedicated to making students the citizens of the future, Bhavan's Vivekananda College of Science, Humanities and Commerce is the college, where we enter in as students and pass out as moulded and well- read person, ready to take on the challenges that come by us in the future.

INDEX

S. No.	TOPIC	Page Number
1.	Abstract	6
2.	Problem Statement	7-8
3.	Proposed System	9-11
3.1	Overview & Goals	9-10
3.2	Modules	10-11
4.	System Requirement	12-14
4.1	Software Requirement	12-13
4.2	Hardware Requirement	14
5	Software Used	15-18
6	System Analysis	19
6.1	Entity-Relationship Diagram	19-27
7	System Design	28
7.1	Class Diagram	29-36
7.2	Activity Diagram	37-44
7.3	Use Case Diagram	45-48
7.4	Sequence Diagram	49-53
7.5	Deployment Diagram	54-56
7.6	Component Diagram	57-61
7.7	State Chart Diagram	62-66
8	Frontend Screens	67-70
9	Backend Screens	71-73
10	Sample Code	74-78
11	Testing	79-81
12	Implementation	82
13	Maintenance	83
14	Conclusion	84
15	Bibliography	85

1.ABSTRACT

Author's Haven – A Comprehensive Writing and World-Building Platform is intended to offer an advanced and integrated platform for writers to efficiently create, organize, and develop their work.

Author's Haven is an interactive web-based platform that provides a comprehensive and user-friendly space for writers to create manuscripts, characters, plot ideas, and story relationships. Developed using React.js for the frontend and Firebase services for backend functionality, the platform ensures real-time data synchronization, secure storage, and a smooth user experience across all devices.

The platform provides necessary modules such as user login, project management, chapter writing using a rich-text editor, character management, relationship mapping using interactive flowcharts, and organized world-building tools. An AI-powered research assistant using OpenAI or Google Gemini API is integrated to offer idea generation, rewriting assistance, and precise research help to boost creativity and productivity.

2.PROBLEM STATEMENT

In the ever-changing creative and digital environment, authors tend to use a variety of isolated tools for writing, character creation, research, and story development. The isolated process causes disorganization, decreased productivity, and makes it difficult to maintain consistency from project to project. Handling story components manually or using different tools is a challenge for authors in terms of productivity, organization, and data security. Author's Haven will solve this problem by offering a comprehensive, intelligent, and cloud-based writing and world-building solution.

2.1. KEY CHALLENGES:

User Engagement and Experience:

Issue: Current writing solutions do not have an integrated and interactive environment designed specifically for authors and creative professionals.

Impact: Constantly switching between applications causes workflow interruptions, decreased focus, and decreased creative productivity.

Fragmented Story and Character Management:

Issue: Current solutions lack an integrated space for handling chapters, character information, relationship organization, timelines, and world-building notes.

Impact: Unorganized information leads to a disjointed story, character growth, and organized plot development.

Inefficient Research and Idea Development:

Issue: Writers are forced to rely on third-party applications for research, idea development, and content optimization.

Impact: Frequent context switching hinders the writing process and disrupts the creative flow.

Data Management and Real-Time Synchronization:

Issue: Organizing files manually and the absence of a centralized cloud solution raise the possibility of losing information and getting confused about different versions.

Impact: Writers face challenges in accessing the latest information on multiple devices and organizing backups.

Authentication and Data Security:

Issue: The lack of robust authentication and storage systems can lead to the vulnerable data of creative work being accessed by unauthorized individuals.

Impact: Security concerns lower user trust and could lead to the loss or misuse of intellectual property.

Scalability and System Performance:

Issue: The lack of scalability in traditional writing applications makes them inefficient when handling multiple projects, long manuscripts, and increasing data.

Impact: System performance and sluggishness impact productivity and user experience.

It is imperative to address these issues in order to create a well-rounded writing and world-building application that provides a seamless, secure, scalable, and organized environment for writers and creative professionals.

3.PROPOSED SYSTEM

3.1. OVERVIEW & GOALS:

Author's Haven is an intelligent web-based writing and world-building tool that aims to centralize the entire writing process for authors. The tool is a combination of writing, character management, relationship organization, project management, and AI-assisted research in a single, unified interface. The tool is built using React.js and Firebase services and is primarily focused on providing a seamless, secure, and user-friendly experience for authors.

User-Centric Experience:

Objective: To provide an intuitive and distraction-free writing environment.

Goal: To ensure smooth navigation, real-time auto-saving, and easy access to projects, characters, and planning features.

Integrated Writing and Project Management:

Goal: To implement a structured project management system with chapters, notes, timelines, and world-building areas with centralized cloud storage.

Objective: To create a system that enables authors to organize all aspects of their stories in one place.

AI-Driven Research and Assistance:

Goal: Increase creativity and productivity through the use of intelligent tools.

Objective: Integrate an AI research assistant (OpenAI API or Google Gemini API) for idea development, rewriting assistance, and sound research support.

Visual Story Planning and Relationship Mapping:

Goal: Strengthen story development and character consistency.

Objective: Develop interactive flowcharts for character relationship mapping and visual story linking.

Secure Authentication and Data Privacy:

Goal: Provide safe access and protect user content.

Objective: Integrate Firebase Authentication with role-based access and secure cloud storage for protecting creative content.

Scalability and Performance Optimization:

Goal: Enable the platform to support multiple projects and growing data in an efficient manner.

Objective: Leverage Firebase Firestore and optimized frontend design for maintaining smooth performance and real-time synchronization.

Future Enhancements:

Collaboration tools for co-authoring projects.

Export options such as PDF and EPUB.

Voice-to-text functionality.

Plagiarism checking tools.

With these goals in mind, Author's Haven seeks to change the writing process into an organized, intelligent, scalable, and user-friendly digital environment that enables writers to write efficiently and creatively.

3.2. MODULES:

Admin Module:

User Management: Admin can track registered users and manage platform-level settings.

System Monitoring: Track system usage, data storage, and overall platform performance.

Writer Module:

Project Management: Writers can manage multiple writing projects.

Chapter Writing: Use a rich-text editor to write chapters with auto-save functionality.

Character Management: Create and manage detailed character profiles, including personality, history, appearance, and roles.

Relationship Mapping: Map character relationships using interactive flowcharts.

World-Building and Notes: Manage plot outlines, timelines, location information, and research notes.

AI Research Assistant: Leverage AI-powered tools for idea generation, rewriting, and research assistance.

Authentication Module:

Sign Up and Login: Users can register and log in securely using Firebase Authentication.

Secure Data Access: Ensure that each writer can only access their own projects and data stored securely in cloud storage.

4.SYSTEM REQUIREMENTS

4.1. SOFTWARE:

The software requirements are description of features and functionalities of the target system. Requirements convey the expectations of users from the software product. The requirements can be obvious or hidden, known or unknown, expected or unexpected from client's point of view. The software requirements for this software are as follows:

The software requirements describe the technologies and tools used to design, develop, and deploy Author's Haven. The system is built using a modern MERN stack architecture along with AI integrations and development utilities to ensure performance, scalability, and security.

The software requirements for Author's Haven are as follows:

FRONTEND:

React (Vite): Used to build a fast and interactive single-page application with dynamic UI updates.

Tailwind CSS: Provides modern styling with a clean, premium workspace design.

React Router: Manages navigation between Dashboard, Writing Workspace, and Character sections.

Axios: Handles communication between the frontend and backend API.

Firebase SDK: Manages user authentication (Google Sign-In and Email/Password login).

BACKEND:

Node.js: Server-side runtime environment for handling requests and processing data.

Express.js: Framework used to build RESTful APIs for managing projects, chapters, and characters.

Mongoose (ODM): Defines structured schemas for Projects, Chapters, and Characters and manages communication with MongoDB.

JSON Web Tokens (JWT): Used in protected routes to ensure secure and authorized access to user data.

DATABASE:

MongoDB: NoSQL database used for persistent data storage.

Project-Centric Data Structure: Every data entity (Chapter, Character, AI Research) is linked with a projectId to isolate and manage multiple books efficiently.

AI INTEGRATION:

Groq SDK: Integrated using the llama-3.1-8b-instant model for fast AI-powered research responses.

Custom Prompt Engineering: Designed to format AI responses specifically for writers to maintain clarity and focus.

DEVELOPMENT & ENVIRONMENT TOOLS:

Dotenv: Secures environment variables such as API keys and database connection strings.

Concurrently: Allows frontend and backend servers to run simultaneously using a single development command.

Custom PDF Generator: Enables exporting chapters and AI research into structured PDF documents.

DEVELOPMENT TOOLS:

VS Code: Primary code editor used for development.

Postman: API testing tool.

Git & GitHub: Version control and repository management.

4.2. HARDWARE:

All computer software requires certain hardware components to function efficiently. The following specifications are recommended for development and usage of Author's Haven:

PROCESSOR:

Intel i5/i7 or AMD Ryzen 5/7 (or higher)

RAM:

Minimum 8GB (16GB recommended for smooth development)

STORAGE:

At least 100GB SSD recommended

OPERATING SYSTEM:

Windows 10/11, macOS, or Linux

INTERNET CONNECTION:

Stable broadband connection required for database access and AI integration.

5.SOFTWARE USED

The technology used for the development of Author's Haven plays a crucial role in its success. The technologies chosen for the development of Author's Haven should be able to meet the requirements of the system. While selecting the technologies for the development of Author's Haven, enterprise-level concepts were taken into consideration. There is no such thing as the "best" technology; rather, the best technologies were chosen based on performance, scalability, and maintainability. Moreover, instead of reinventing the wheel, established technologies were used.

4.1 SOFTWARE REQUIREMENTS

React.js (Vite):

React.js was employed as the primary framework for building Author's Haven, a dynamic web application. The framework allows for the development of reusable components and fast rendering with a virtual DOM. React played a crucial role in the development of key components such as the Dashboard, Writing Workspace, Character Management page, Relationship Mapping page, and AI Research panel. The framework also allows for smooth API integration with Axios, ensuring efficient communication between the frontend and backend.

Tailwind CSS (for UI Styling):

Tailwind CSS was employed for the design of the user interface using a utility-first design approach. The framework enabled the development of responsive, clean, and modern UI components with a reduced amount of custom code. Tailwind CSS enabled the design of

navigation panels, writing cards, forms, buttons, dashboards, and layouts. The flexibility of the framework ensured a consistent look and feel for different modules, enhancing usability and ensuring a premium look and feel.

CSS (for Additional Styling):

Though Tailwind CSS took care of most of the styling, traditional CSS was employed for additional styling. CSS was employed for animations, smooth transitions, hover effects, layout modifications, and editor improvements that demanded more nuanced control. This ensured that the platform is not only functional but also interactive and aesthetically pleasing.

Axios:

Axios is a JavaScript library employed for managing HTTP requests between the frontend and backend. It makes it easy to send and receive data from the Node.js and Express server. Axios is employed to retrieve projects, store chapters, update characters, retrieve responses for AI research, and delete records from the database. It enables easy JSON data management and error handling, ensuring seamless API communication.

Backend Software

Node.js:

Node.js is used for creating the server-side environment for application logic processing. It handles requests such as saving chapters, getting characters, and managing AI research data.

Express.js:

Express.js is used for creating RESTful APIs. It handles routing, middleware, and request-response processing for the application.

Mongoose (ODM):

Mongoose is used for creating structured schemas for Projects, Chapters, Characters, and AI Research data. It handles proper data validation and storage in the MongoDB database.

JSON Web Tokens (JWT):

JWT is used for securing protected routes. It ensures that only authenticated users can modify their projects and data.

MongoDB:

MongoDB is employed as the main NoSQL database for persistent storage. All user data, including projects, chapters, characters, and AI searches, are stored securely. A project-centric isolation scheme is also enforced where every data entry is associated with a projectId to avoid the combination of multiple books.

Development Tools:

Visual Studio Code (VS Code):

VS Code was employed as the main development tool. It offers code completion, debugging capabilities, integrated terminal support, and extension support such as ESLint and Prettier for maintaining clean and consistent code.

Git and GitHub:

Employed for version control and source code management.

Postman:

Employed for API testing and debugging backend routes.

Dotenv:

Employed to securely manage environment variables such as MongoDB connection strings and AI API keys.

Concurrently:

Employed to run both frontend and backend servers concurrently during development using a single command.

4.2 HARDWARE REQUIREMENTS

Every software application has some hardware requirements for smooth functioning. Below are the hardware requirements for the development and deployment of Author's Haven.

Development Computer:

A computer with a minimum of 8GB RAM, Intel Core i5 processor, and a 512GB SSD for smooth development and testing.

Client Computer (User Access):

Any modern computer or laptop with a minimum of 4GB RAM, an updated web browser (Chrome, Edge, or Firefox), and a stable internet connection.

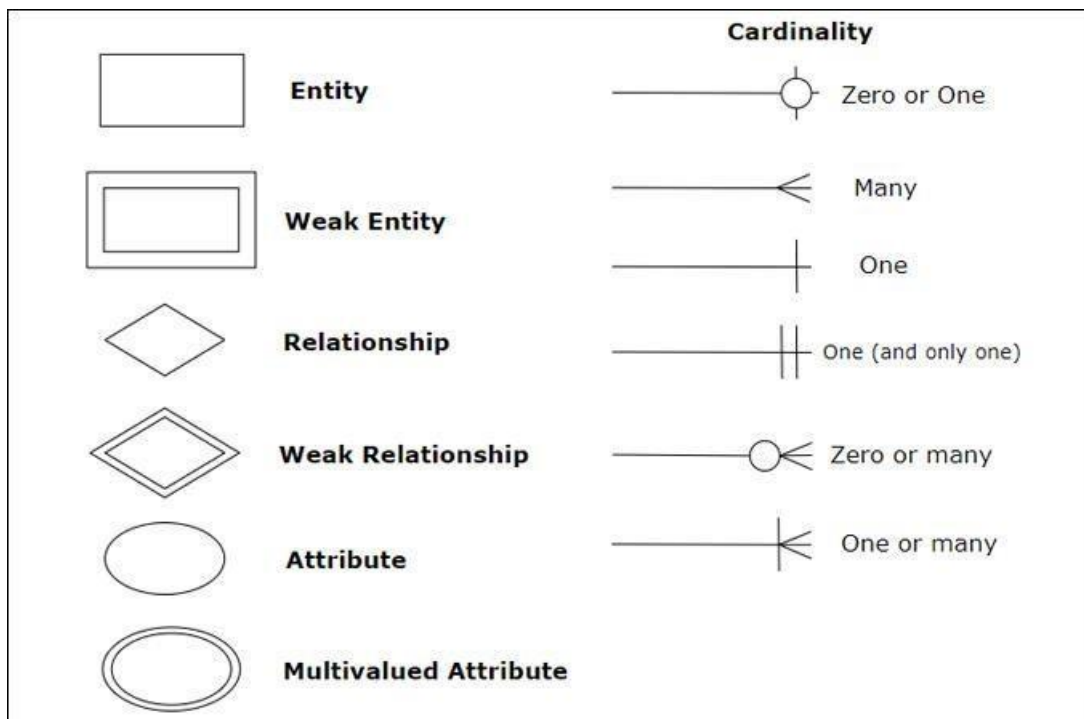
Server Requirements:

A cloud or local server with a minimum of 16GB RAM, Intel Xeon or Ryzen 5 processor (or equivalent), and a 512GB SSD for hosting the Node.js server and MongoDB database efficiently.

6. SYSTEM ANALYSIS

6.1 ENTITY-RELATIONSHIP DIAGRAM:

An Entity Relationship Diagram (ERD) is a visual representation of different entities within a system and how they relate to each other. They are widely used to design relational databases. The entities in the ER schema become tables, attributes and converted the database schema. Entity relationship diagrams are used in software engineering during the planning stages of the software project. They help to identify different system elements and their relationships with each other. Entity-Relationship Diagram (ERD) Symbols and Notations:



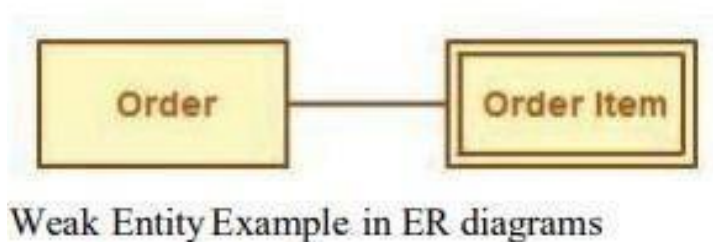
Elements in ER diagrams have three basic elements: entity, attribute, relationship. There are more elements that are based on the main elements. They are weak entity, multi valued attribute, derived attribute, weak relationship, and recursive relationship. Cardinality and ordinality are two other notations used in ER diagrams to further define relationships.

Entity

An entity can be a person, place, event, or object that is relevant to a given system. For example, a school system may include students, teachers, major courses, subjects, fees, and other items. Entities are represented in ER diagrams by a rectangle and named using singular nouns.

Weak Entity

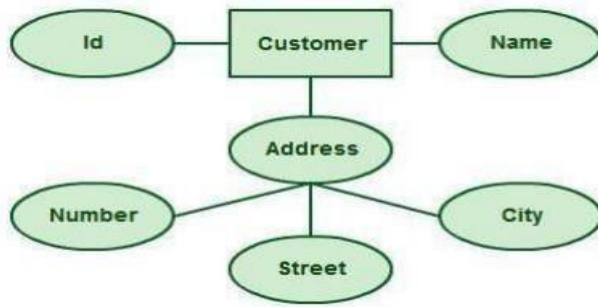
A weak entity is an entity that depends on the existence of another entity. In terms that are more technical it can be defined as an entity that cannot be identified by its own attributes. It uses a foreign key combined with its attributed to form the primary key. An entity like order item is a good example for this. The order item will be meaningless without an order so it depends on the existence of the order. Weak Entity Example in ER diagrams.



Weak Entity Example in ER diagrams

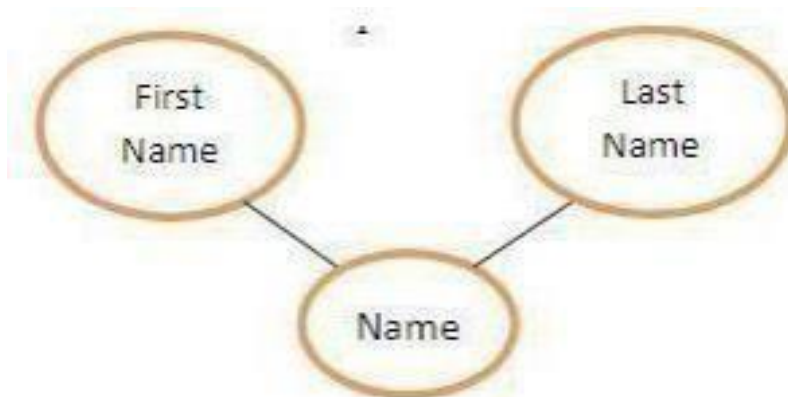
1)Attribute

An attribute is a property, trait, or characteristic of an entity, relationship, or another attribute. For example, the attribute Inventory Item Name is an attribute of the entity Inventory Item. An entity can have as many attributes as necessary. Meanwhile, attributes can also have their own specific attributes. For example, the attribute “customer address” can have the attributes number, street, city, and state. These are called composite attributes. Note that some top level ER diagrams do not show attributes for the sake of simplicity. In those that do, however, oval shapes represent attributes.



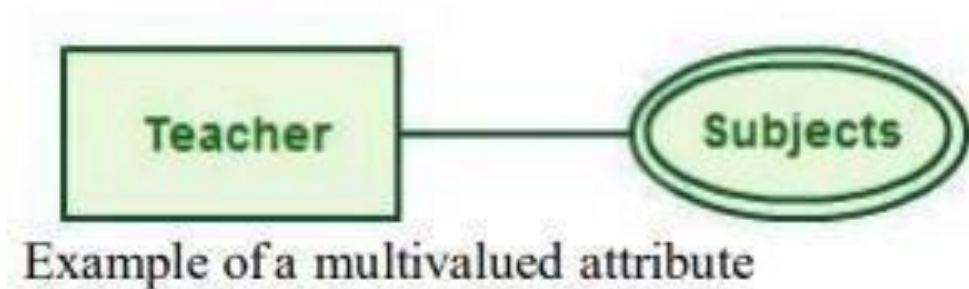
Simple attribute: - An attribute is classified as a simple attribute if it cannot be partitioned into smaller components. For example, age and sex of a person. A simple attribute is represented by an oval.

Composite attribute: - A composite attribute can be subdivided into smaller components, which further form attributes. For example, 'name' attribute of an entity "person" can be broken down into first name and last name, which further form attributes. Grouping of these related attributes forms a composite attribute 'name' is the composite attribute in this example.

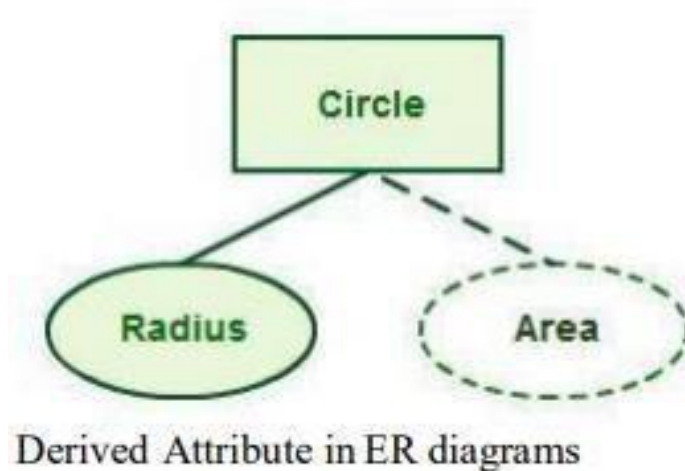


Single valued attribute: - If an attribute of a particular entity represents single value for each instance, then it is called a single-valued attribute. For example, Ramesh, Kamal and Suraj are the instances of entity 'student' and each of them is issued a separate roll number. A single oval is used to represent this attribute.

Multivalued Attribute: - If an attribute can have more than one value, it is called a multivalued attribute. It is important to note that this is different from an attribute having its own attributes. For example, a teacher entity can have multiple subject values. Example of a multivalued attribute.



Derived Attribute: - An attribute based on another attribute. This is found rarely in ER diagrams. For example, for a circle, the area can be derived from the radius. Derived Attribute in ER diagrams.



Relationship: - A relationship describes how entities interact. For example, the entity “Carpenter” may be related to the entity “table” by the relationship “builds” or “makes”. Relationships are represented by diamond shapes and are labelled using verbs.

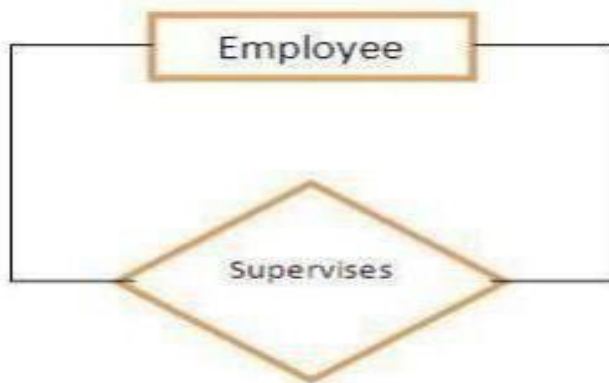
Various terms related to relationships:



Degree of relationship: - It signifies the number of entities involved in a relationship.

Degree of a relationship can be classified into following types:-

Unary relationship: - If only single entity is involved in a relationship, then it is a unary relationship. For example, an employee (manager) supervises another employee.



Binary relationships: - When two entities are associated to form a relation, then it is known as a binary relationship. For example, a person works in a company. Most of the times we use only binary relationship in an e-r diagram. The teacher-student example shown above signifies a binary relationship. Other types of relationships are ternary and quaternary. As the name signifies, a ternary relationship is associated with three entities and a quaternary relationship is associated with four entities.

Connectivity of a relationship: - Connectivity of a relationship describes how many instances of one entity type are linked to how many instances of another entity type. Various categories of connectivity of a relationship are-

One to One (1:1): - “Student allotted a project” signifies a one-to-one relationship because only one instance of an entity is related with exactly one instance of another entity type.



One to Many (1:M): - “A department recruits’ faculty” is a one-to-many relationship because a department can recruit more than one faculty, but a faculty member is related to only one department.

Many to One (M:1): - “Many houses are owned by a person” is a many-to-one relationship because a person can own many houses but a particular house is owned only by a person.

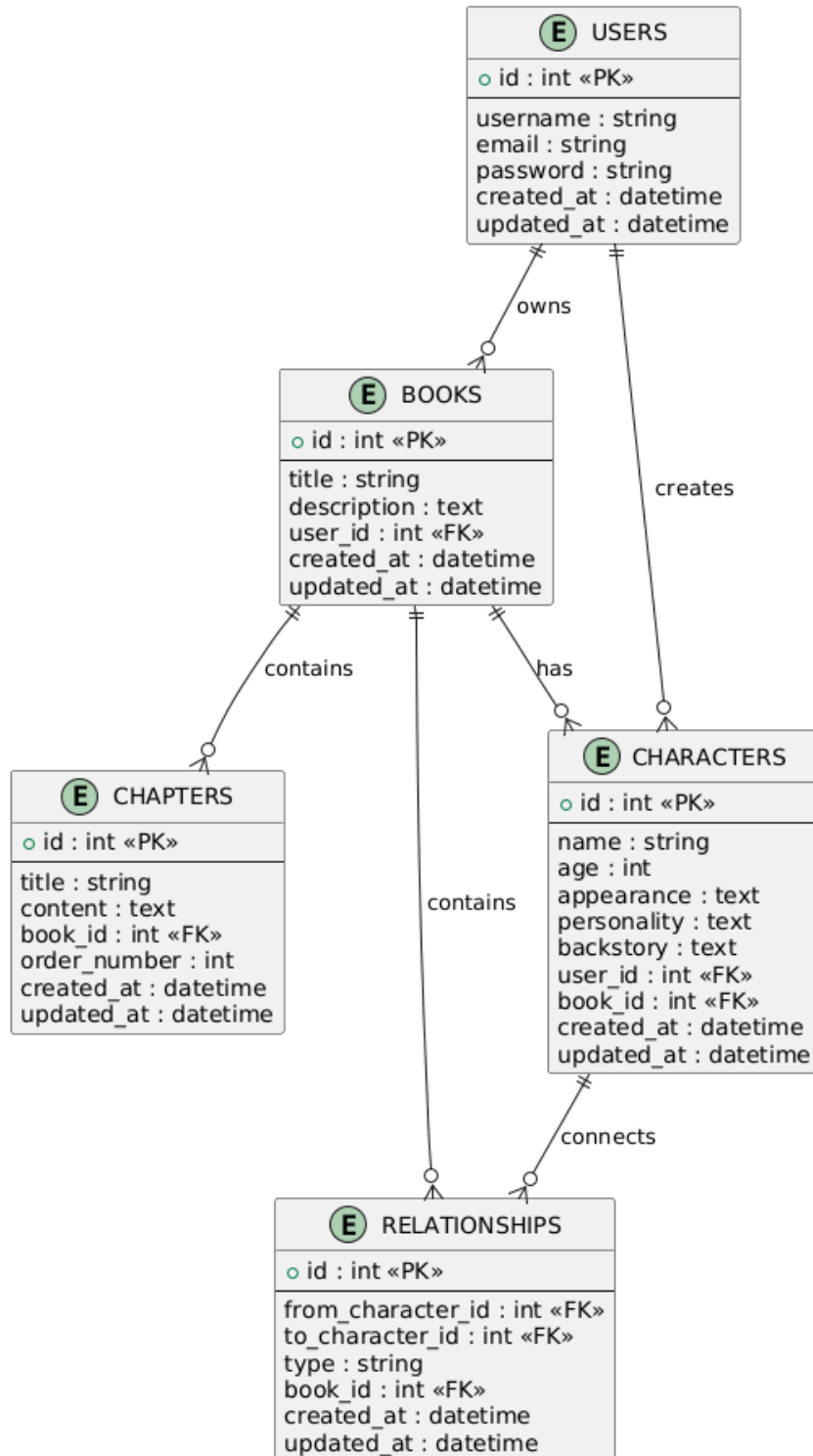


Many to Many (M:N): – “Author writes books” is a many-to-many relationship because an author can write many books and a book can be written by many authors.



ER DIAGRAM FOR AUTHOR'S HAVEN:

ER Diagram - Author's Haven



The ER diagram shows the data structure of Author's Haven by pointing out the main entities, their attributes, and relationships. This diagram gives a conceptual view of how user data and writing content are organized in the system.

Entities and Attributes:

1. Users

Attributes: id (Primary Key), username, email, password, created_at, updated_at

A user may have many books.

2. Books

Attributes: id (Primary Key), title, description, user_id (Foreign Key), created_at, updated_at

A book belongs to a user and holds chapters and characters.

3. Chapters

Attributes: id (Primary Key), title, content, book_id (Foreign Key), order_number, created_at, updated_at

A book holds many chapters in order.

4. Characters

Attributes: id (Primary Key), name, age, appearance, personality, backstory, user_id (Foreign Key), book_id (Foreign Key), created_at, updated_at

A character is in a particular book.

5. Relationships

Attributes: id (Primary Key), from_character_id (Foreign Key), to_character_id (Foreign Key), type, book_id (Foreign Key), created_at, updated_at

Shows the relationships between characters in a book.

Relationships:

One user may own many books.

One book may have many chapters and characters.

Characters are related to each other through relationships in the same

7. SYSTEM DESIGN

UML DIAGRAMS:

UML stands for Unified Modeling Language. This object-oriented system of notation has evolved from the work of Grady Brooch, James Rumbaugh, Ivar Jacobson and the rational software corporation. These renowned computer scientists fused their respective technologies into a single, standardized model. Today, the Object Management Group (OMG) as the standard for modeling object-oriented concepts. The unified modeling notation that is governed by a set of syntactic semantic and pragmatic rules. A UML system is represented using five different views that describe the system from a distinctly different perspective. Each view is defined by a set of diagrams, which is as follows.

Types of UML Diagram we drew in this project are:

- Class Diagram
- Activity Diagram
- Use Case Diagram
- Sequence Diagram
- Component Diagram
- Deployment Diagram
- State Chart Diagram

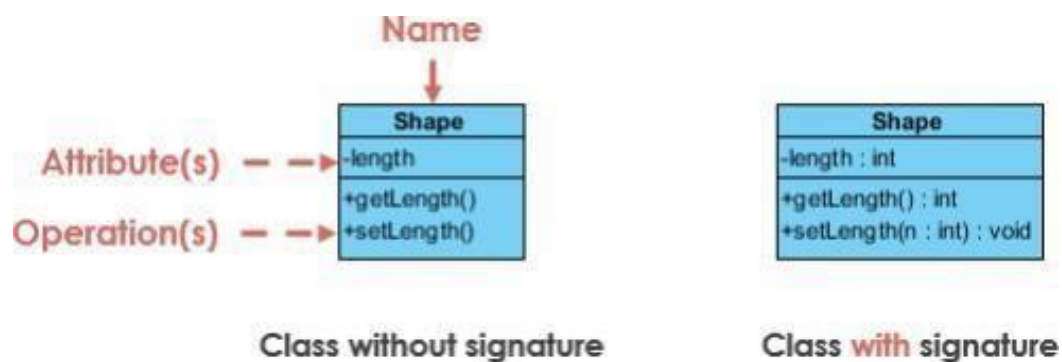
7.1 CLASS DIAGRAM

The UML Class diagram is a graphical notation used to construct and visualize object-oriented systems. A class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the systems:

- Classes
- their attributes,
- operations (or methods), • and the relationships among objects.

Class: A Class is a blueprint for an object. Objects and classes go hand in hand. We cannot talk about one without talking about the other. In addition, the entire point of Object-Oriented Design is not about objects, it is about classes, because we use classes to create objects. Therefore, a class describes what an object will be, but it is not the object itself.

UML Class Notation: A class represents a concept, which encapsulates state (attributes) and behaviour (operations). Each attribute has a type. Each operation has a signature. The class name is the only mandatory information.

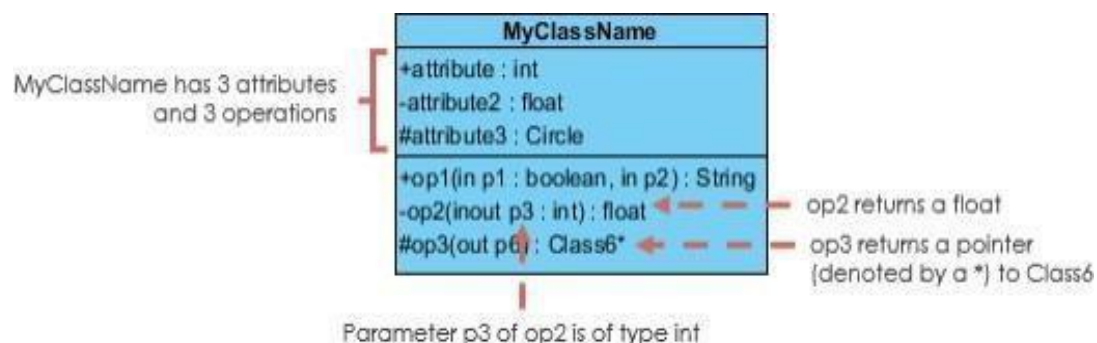


Class Name: The name of the class appears in the first partition.

Class Attributes: Attributes are shown in the second partition. The attribute type is shown after the colon. • Attributes map onto member variables (data members) in code.

Class Operations (Methods):

- Operations are shown in the third partition. They are services the class provides.
- The return type of a method is shown after the colon at the end of the method signature.
- The return type of method parameters are shown after the colon following the parameter name. • Operations map onto class methods in code.



Class Visibility, the + , - and # symbols before an attribute and operation name in a class denote the visibility of the attribute and operation.

+ denotes public attributes or operations

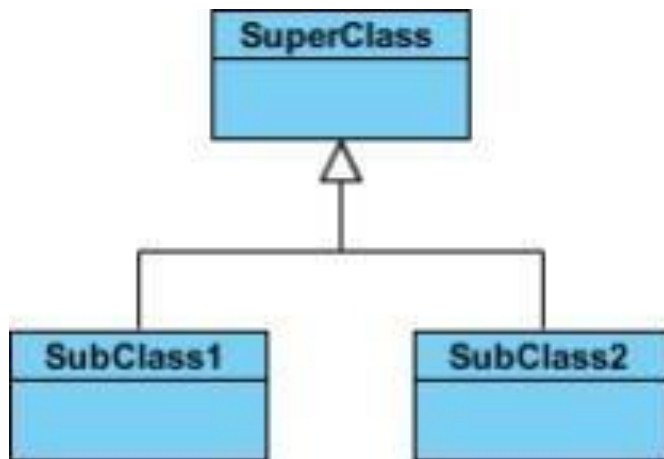
- denotes private attributes or operations • # denotes protected attributes or operations.

Relationships between classes:

Inheritance (or Generalization):

A generalization is a relationship between a more general classifier and a more specific classifier. Each instance of the specific classifier is also an indirect instance of the general classifier. Thus, the specific classifier inherits the features of the more general classifier. The

figure below shows an example of inheritance hierarchy. SubClass1 and SubClass2 are derived from Superclass. The relationship is displayed as a solid line with a hollow arrowhead that points from the child element to the parent element.



Association:

Associations are relationships between classes in a UML Class Diagram. A solid line between classes represents them. Associations are typically named using a verb or verb phrase, which reflects the real world problem domain.

Simple Association

- A structural link between two peer classes.
- There is an association between Class1 and Class2



Cardinality:

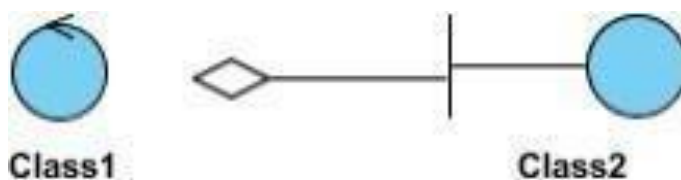
Cardinality is expressed in terms of:

- one to one
- one to many
- many to many

Aggregation

- A special type of association.
- It represents a "part of" relationship.
- Class2 is part of Class1.
- Many instances (denoted by the *) of Class2 can be associated with Class1. • Objects of Class1 and Class2 have separate lifetimes.

The figure below shows an example of aggregation. The relationship is displayed as a solid line with an unfilled diamond at the association end, which is connected to the class that represents the aggregate.

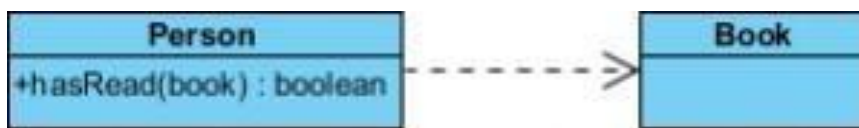


Composition

- A special type of aggregation where parts are destroyed when the whole is destroyed.
- Objects of Class2 live and die with Class1. • Class2 cannot stand by itself.

Dependency

- An object of one class might use an object of another class in the code of a method.
- If the object is not stored in any field, then this is modeled as a dependency relationship.
- A special type of association.
- Exists between two classes if changes to the definition of one may cause changes to the other (but not the other way around). • Class1 depends on Class2.



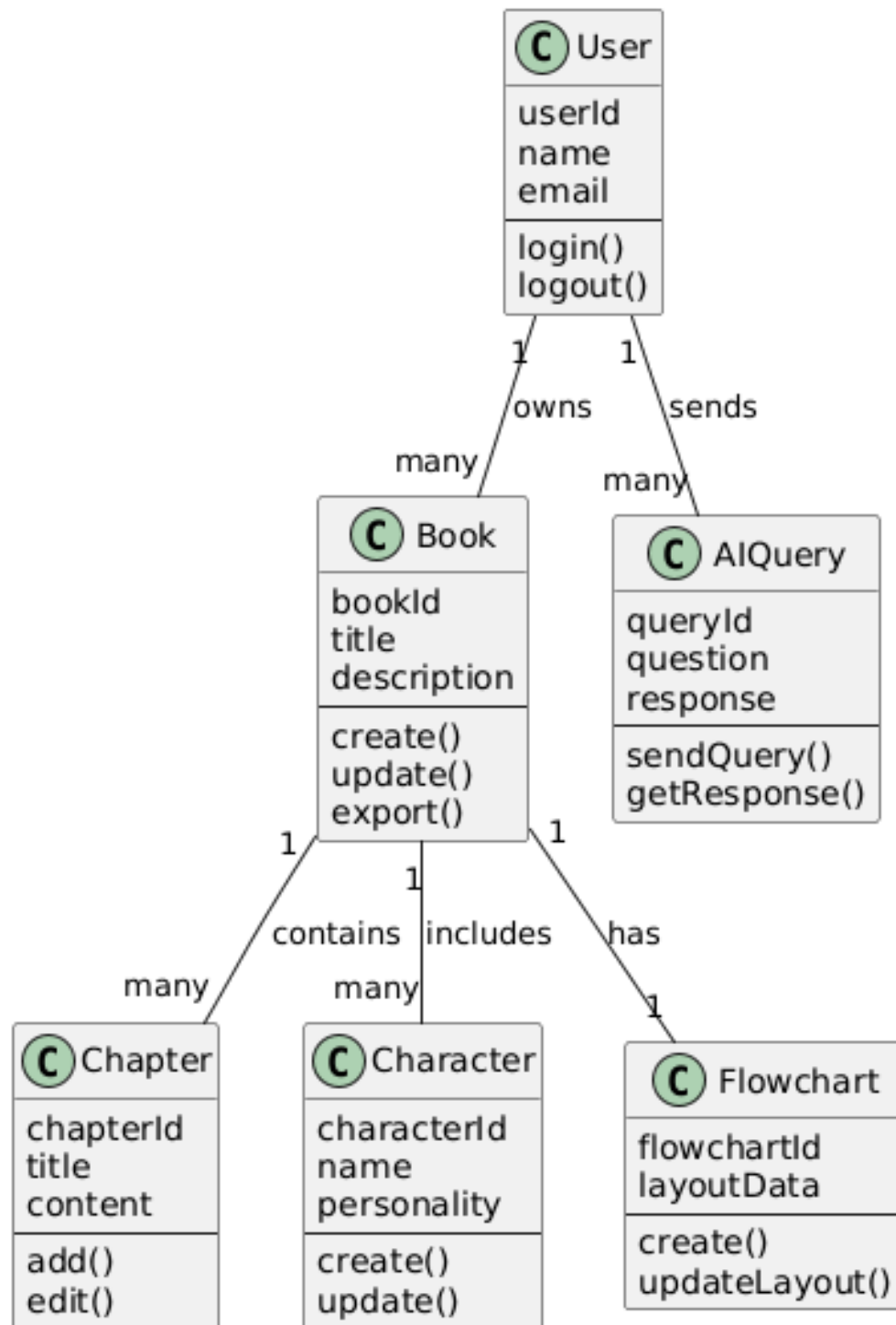
The figure below shows an example of dependency. The relationship is displayed as a dashed line with an open arrow. The figure above shows another example of dependency. The Person class might have a has Read method with a Book parameter that returns true if the person has read the book (perhaps by checking some database).

Realization:

Realization is a relationship between the blueprint class and the object containing its respective implementation level details. This object is said to realize the blueprint class. In other words, you can understand this as the relationship between the interface and the implementing class. For example, the Owner interface might specify methods for acquiring property and disposing of property. The Person and Corporation classes need to implement these methods, possibly in very different ways.

CLASS DIAGRAM FOR AUTHOR'S HAVEN:

Class Diagram - Author's Haven



The class diagram represents the object-oriented structure of Author's Haven. It shows the main classes, their attributes, methods, and relationships within the system.

Classes and Relationships:

User Class:

Attributes: userId, name, email

Methods: login(), logout()

A User can own multiple Books and send multiple AI Queries.

Book Class:

Attributes: bookId, title, description

Methods: create(), update(), export()

A Book belongs to one User and contains multiple Chapters and Characters. It also includes a Flowchart for relationship mapping.

Chapter Class:

Attributes: chapterId, title, content

Methods: add(), edit()

A Book can have multiple Chapters.

Character Class:

Attributes: characterId, name, personality

Methods: create(), update()

A Book can include multiple Characters.

Flowchart Class:

Attributes: flowchartId, layoutData

Methods: create(), updateLayout()

Each Book has one Flowchart to visually represent character relationships.

AIQuery Class:

Attributes: queryId, question, response

Methods: sendQuery(), getResponse()

A User can send multiple AI queries for research and idea generation.

Relationships:

One User can own many Books.

One User can send many AI Queries.

One Book can contain many Chapters and Characters.

One Book has one Flowchart for visual mapping.

7.2 ACTIVITY DIAGRAM

Activity diagram is another important diagram in UML to describe the dynamic aspects of the system. Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent. Activity diagrams deal with all type of flow control by using different elements such as fork, join, etc. Purpose of Activity Diagrams: The basic purpose of activity diagrams is similar to other four diagrams. It captures the dynamic behaviour of the system. Other four diagrams are used to show the message flow from one object to another but the activity of the diagram is used to show the message flow from one activity to another. Activity is a particular operation of the system. Activity diagrams are not only used for visualizing the dynamic nature of a system, but they are also used to construct the executable system by using forward and reverse engineering techniques. The only missing thing in the activity diagram is the message part.

The purpose of an activity diagram can be described as-

- Draw the activity flow of a system.
- Describe the sequence from one activity to another.
- Describe the parallel, branched and concurrent flow of the system.

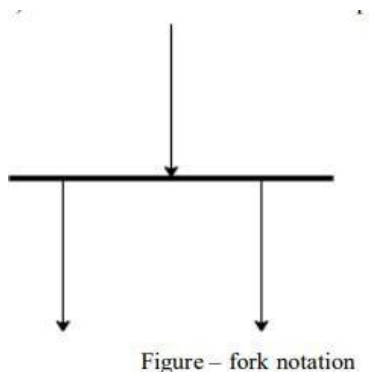
Components of an Activity Diagram: -

- Initial State or Start Point:** A small filled circle followed by an arrow represents the initial action state or the start point for any activity diagram.



- ii. **Activities:** The categorization of behaviour into one or more actions is termed as an activity. In other words, it can be said that an activity is a network of nodes that are connected by edges. The edges depict the flow of execution. It may contain action nodes, control nodes, or object nodes. The control flow of activity is represented by control nodes and object nodes that illustrate the objects used within an activity. The activities are initiated at the initial node and are terminated at the final node.

- iii. **Fork** – Fork nodes are used to support concurrent activities.



When we use a fork node when both the activities get executed concurrently i.e., no decision is made before splitting the activity into two parts. Both parts need to be. We use a rounded solid rectangular bar to represent a Fork notation with incoming arrows from the parent activity state and outgoing arrows towards the newly created activities. For example: In the example below, the activity of making coffee can be split into two concurrent activities and hence we use the fork notation.

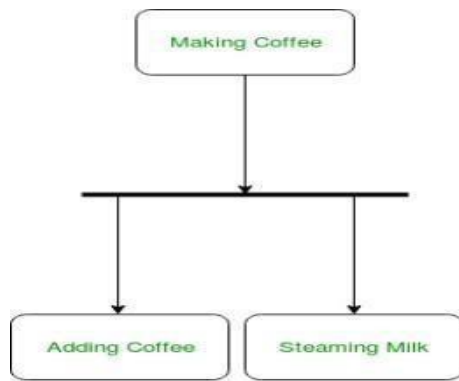


Figure – a diagram using fork

- iv. **Join** - Join nodes are used to support concurrent activities converging into one. For join notations, we have two or more incoming edges and one outgoing edge.

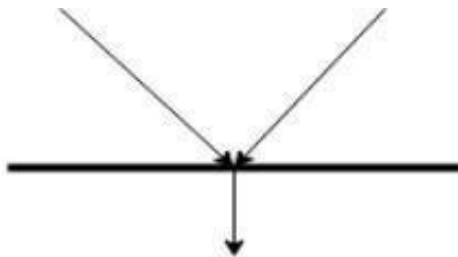


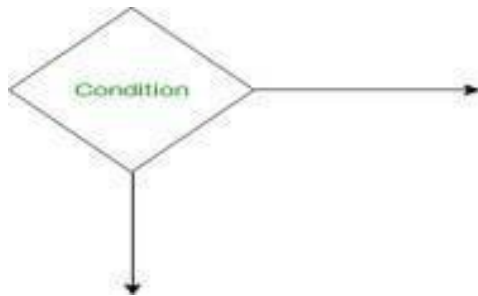
Figure – join notation

For example – When both activities i.e. steaming the milk and adding coffee get completed, we converge them into one final activity.

- v. **Action Flow** - Action flows, also called edges and paths, illustrate the transitions from one action state to another. They are usually drawn with an arrowed line.

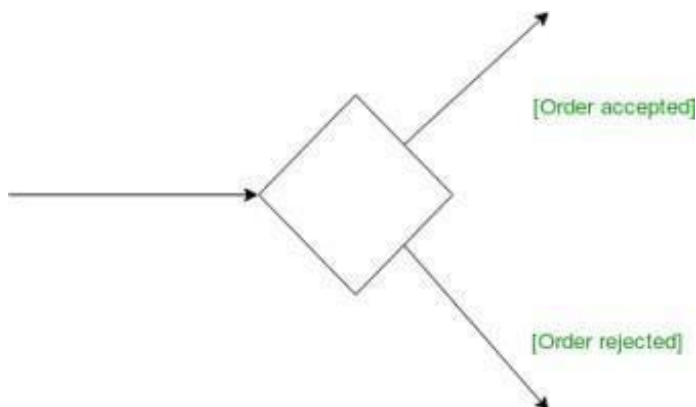


- vi. Decisions and Branching** - When we need to make a decision before deciding the flow of control, we use the decision node.

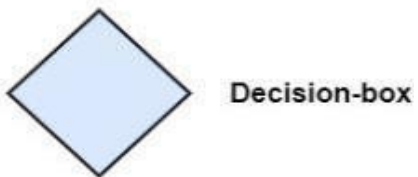
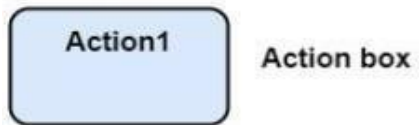


The outgoing arrows from the decision node can be labelled with conditions or guard expressions. It always includes two or more output arrows.

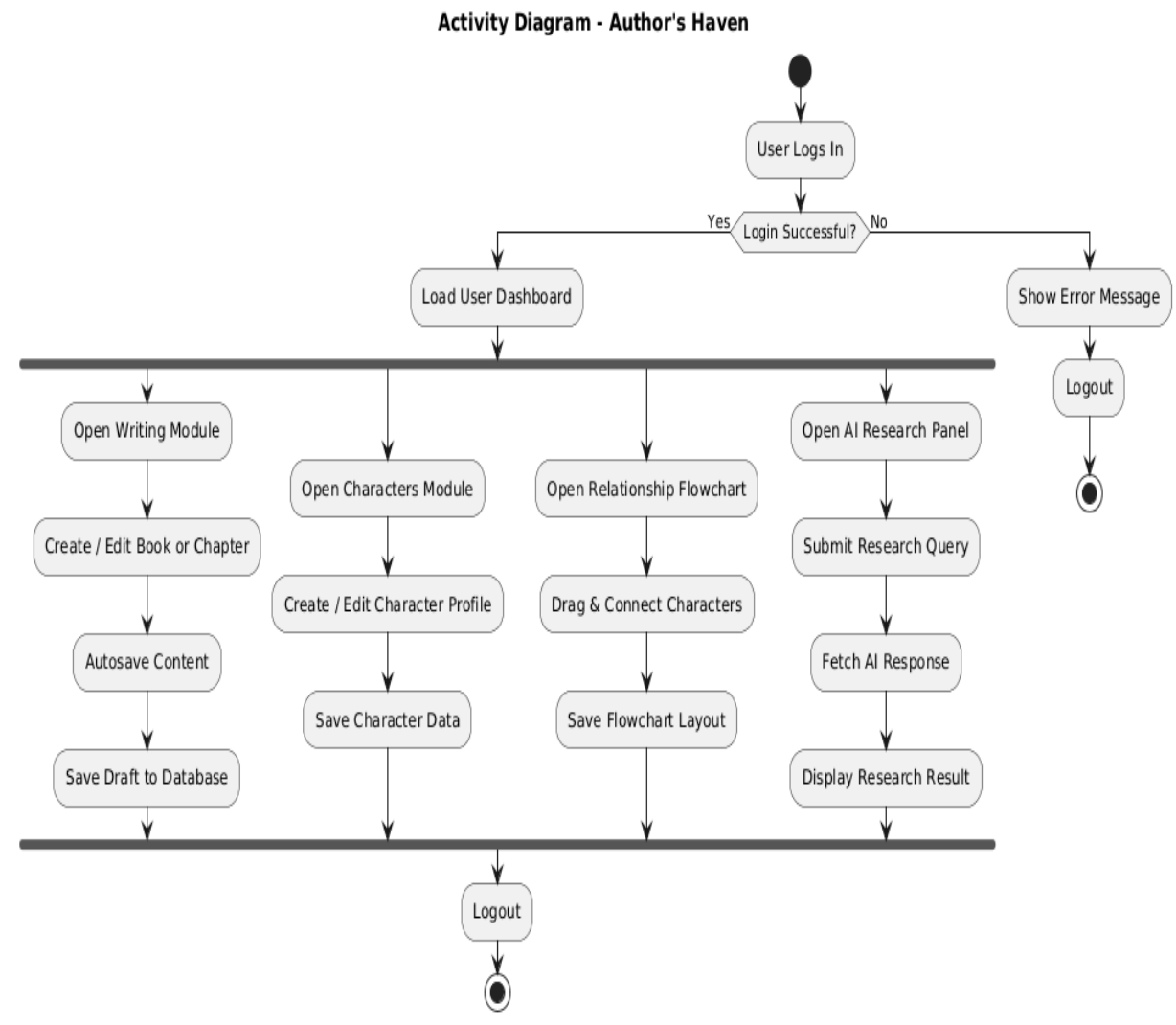
- vii. Guards** - A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets.



- viii. Final State or End Point** - An arrow pointing to a filled circle nested inside another circle represents the final action state.



ACTIVITY DIAGRAM FOR AUTHOR'S HAVEN:



Start Node:

The black circle at the top marks the start of the workflow.

User Login Process:

The user logs into the system.

If the login is successful, the user dashboard is loaded.

If the login fails, an error message is shown, and the user can logout.

Dashboard Actions:

After successful login, the user can perform various actions on the dashboard:

1. Writing Module:

The user accesses the writing module.

The user creates or edits a book or chapter.

The content is automatically saved.

The draft is stored in the database.

2. Characters Module:

The user accesses the characters module.

The user creates or edits character profiles.

The character data is saved to the database.

3. Relationship Flowchart:

The user accesses the relationship flowchart.

The user drags and connects characters.

The flowchart layout is saved.

4. AI Research Panel:

The user accesses the AI research panel.

The user submits a research query.

The system retrieves the AI response.

The result is shown to the user.

End Process:

After finishing all activities, the user can logout, thus ending the workflow.

7.3 USE-CASE DIAGRAM

In Unified Modelling Language (UML), use case diagrams are used to show the interactions between actors and a system. The system can be a website or another similar entity being developed. An actor is a person or entity that interacts with the system - normally, this is a user. In essence, use case diagrams depict the system, who is using it, how they are using it, or what they are doing. On a macro scale, the information you can glean from use case diagrams allow you to set the scope of your system or application and what goals it can help the actors achieve.

Use case diagrams are typically used in the following scenarios:

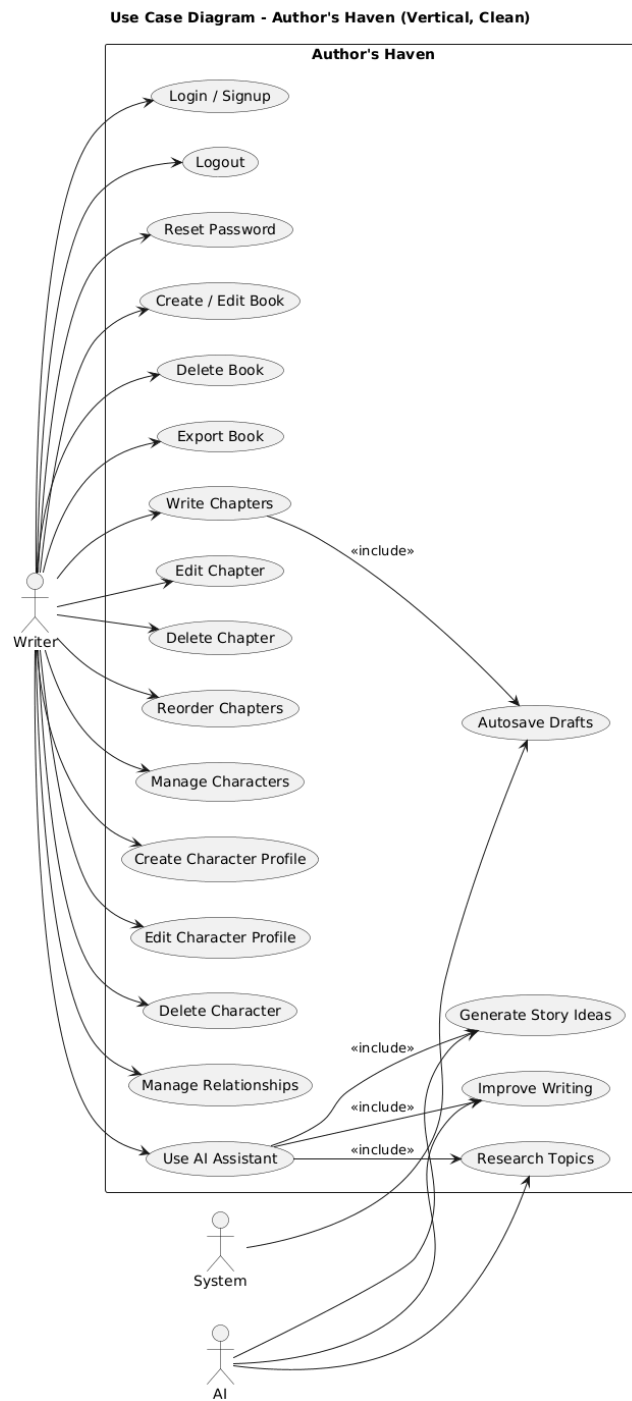
- Depicting the scope and goal of the interactions between the actor and the system.
- Modelling the interactions between actors and the system, as well as the general flow of events.
- Identifying internal or external factors that might influence the outcome of interaction and, by extension, the system itself.

Use Case Diagram Symbols:

The notation for a use case diagram is straightforward and does not involve as many types of symbols as other UML diagrams.

- **Use cases:** Horizontally shaped ovals that represent the different uses that a user might have.
- **Actors:** Stick figures that represent the people actually employing the use cases.
- **Relationship:** A line between actors and use cases. In complex diagrams, it is important to know which actors are associated with which use cases.

USECASE DIAGRAM FOR AUTHOR'S HAVEN:



1. Actor

Writer:

The Writer is the main user of the system. The writer uses all the major functionalities like book, chapter, character, relationship management, and the AI assistant.

System:

It represents the internal system processes like autosave drafts.

AI:

It represents the external AI service used for generating story ideas, improving writing, and researching topics.

2. Use Cases (Inside the System Boundary)

The rectangle represents the system boundary of Author's Haven. Each oval inside the rectangle represents a functionality provided by the system.

Main Use Cases:

Login / Signup

Logout

Reset Password

Create / Edit Book

Delete Book

Export Book

Write Chapters

Edit Chapter

Delete Chapter

Reorder Chapters

Manage Characters

Create Character Profile

Edit Character Profile

Delete Character

Manage Relationships

Use AI Assistant

Included Functionalities:

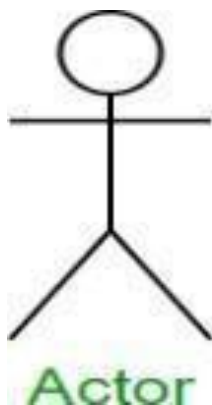
Write Chapters includes Autosave Drafts.

Use AI Assistant includes Generate Story Ideas, Improve Writing, and Research Topics.

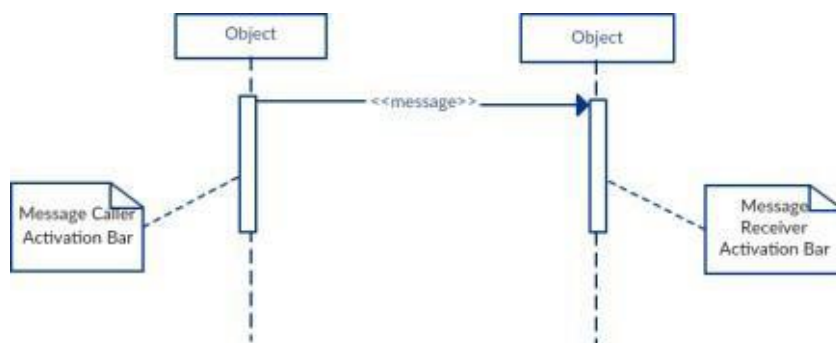
7.4 SEQUENCE DIAGRAM

Sequence diagrams describe interactions among classes in terms of an exchange of messages over time. They are also called event diagrams. A sequence diagram is a good way to visualize and validate various runtime scenarios. These can help to predict how a system will behave and to discover responsibilities a class may need to have in the process of modelling a new system.

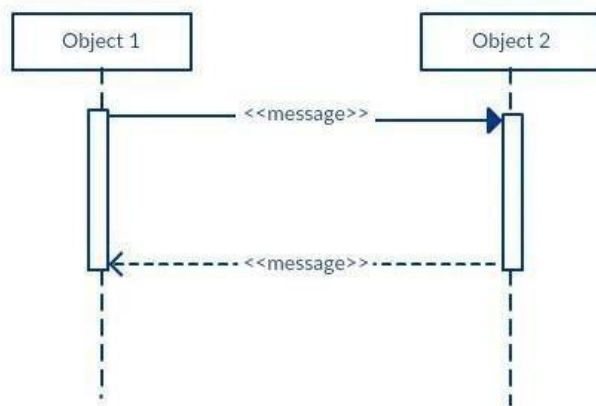
Basic Sequence Diagram Notations:- • **Actor symbol:** Shows entities that interact with or are external to the system.



- **Activation:** Activation boxes represent the time an object needs to complete a task. When an object is busy executing a process or waiting for a reply message, use a thin grey rectangle placed vertically on its lifeline.

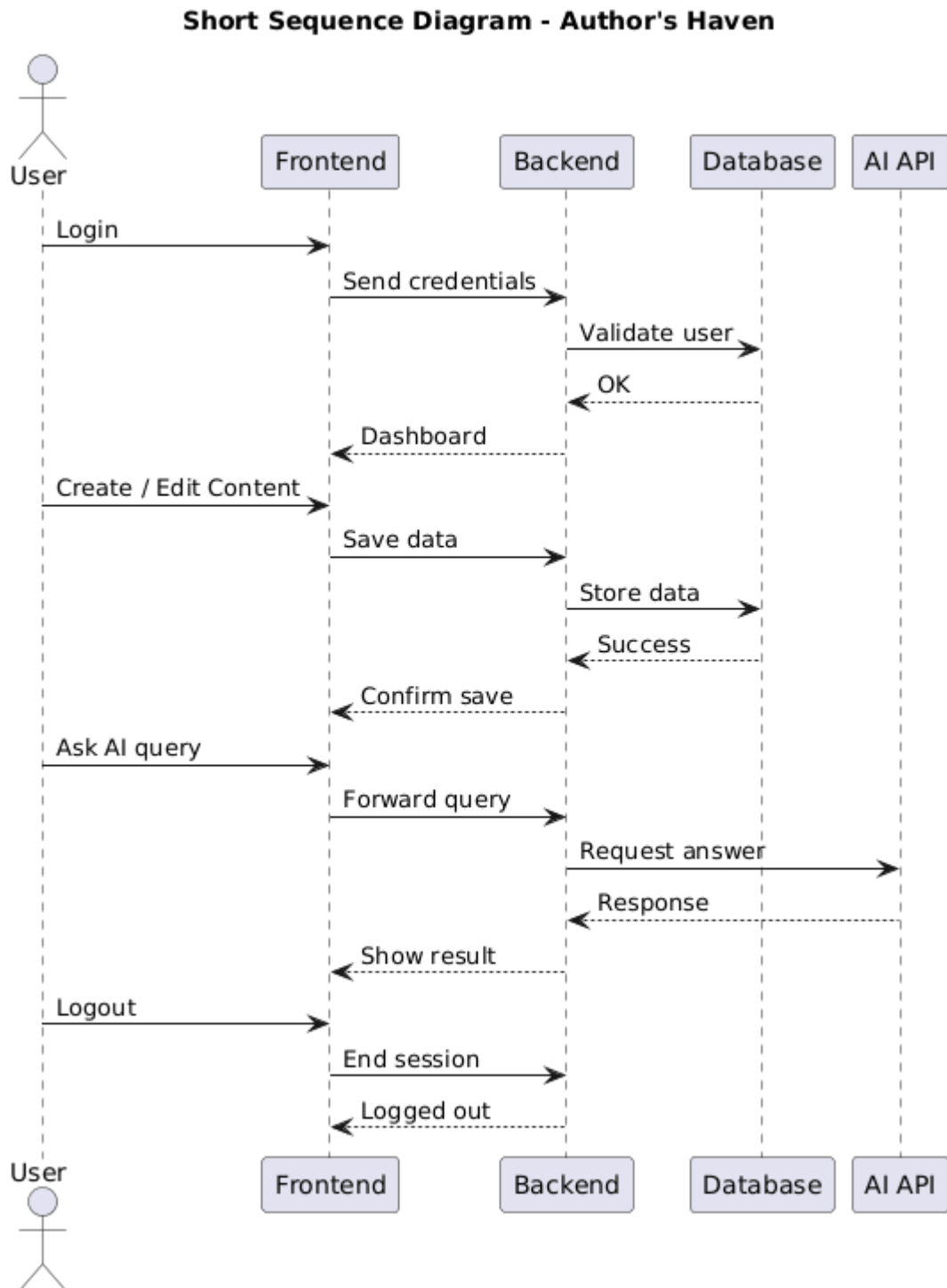


- **Messages:** Messages are arrows that represent communication between objects. We use half-arrowed lines to represent asynchronous messages. Asynchronous messages are sent from an object that will not wait for a response from the receiver before continuing its tasks.
- **Return:** A return message is used to indicate that the message receiver is done processing the message and is returning control over to the message caller. Return messages are optional notation pieces, for an activation bar that is triggered by a synchronous message always implies a return message.



- **Lifelines:** Lifelines are vertical dashed lines that indicate the object's presence over time.
- **Comment:** UML diagrams generally permit the annotation of comments in all UML diagram types. The comment object is a rectangle with a folded-over corner as shown below. The comment can be linked to the related object with a dashed line.

SEQUENCE DIAGRAM FOR AUTHOR'S HAVEN:



Important Elements of Figure:

1. Actors (Stick Figures at the Left Side)

User (Writer, Reader, or Admin) interacts with the system.

2. Lifelines (Vertical Dotted Lines)

Frontend

Backend

Database

AI API

3. Messages (Horizontal Arrows)

Represent interactions between the user and system components.

Explanation of the Workflow:

1. Login Module Workflow:

User logs in by entering credentials.

The system sends credentials from Frontend to Backend.

Backend validates the user with the Database.

If successful:

Dashboard is displayed to the user.

If failed:

Login error message appears.

2. Content Management Workflow:

User decides to create or edit content.

Frontend sends content data to Backend.

Backend stores the content in the Database.

If successful:

Confirmation message is shown to the user.

If failed:

Error message appears.

3. AI Query Workflow:

User makes an AI query.

Frontend forwards the query to Backend.

Backend sends the request to the AI API.

AI API processes the query and returns a response.

If successful:

Result is displayed to the user.

If failed:

Error message appears.

4. Logout Workflow:

User clicks logout.

Frontend sends session termination request to Backend.

Backend ends the session.

If successful:

User is logged out and confirmation is shown.

7.5 DEPLOYMENT DIAGRAM :

A deployment diagram is a UML diagram type that shows the execution architecture of a system, including nodes such as hardware or software execution environments, and the middleware connecting them.

Deployment diagrams are typically used to visualize the physical hardware and software of a system. Using it you can understand how the system will be physically deployed on the hardware. Deployment diagrams help model the hardware topology of a system compared to other UML diagram types which mostly outline the logical components of a system.

Deployment Diagram Notations:

In order to draw a deployment diagram, you need to first become familiar with the following deployment diagram notations and deployment diagram elements.

- **Nodes:** A node, represented as a cube, is a physical entity that executes one or more components, subsystems or executables. A node could be a hardware or software element.

- **Artifacts:** Artifacts are concrete elements that are caused by a development process.

Examples of artifacts are libraries, archives, configuration files, executable files etc.

- **Communication Association:** This is represented by a solid line between two nodes. It shows the path of communication between nodes.

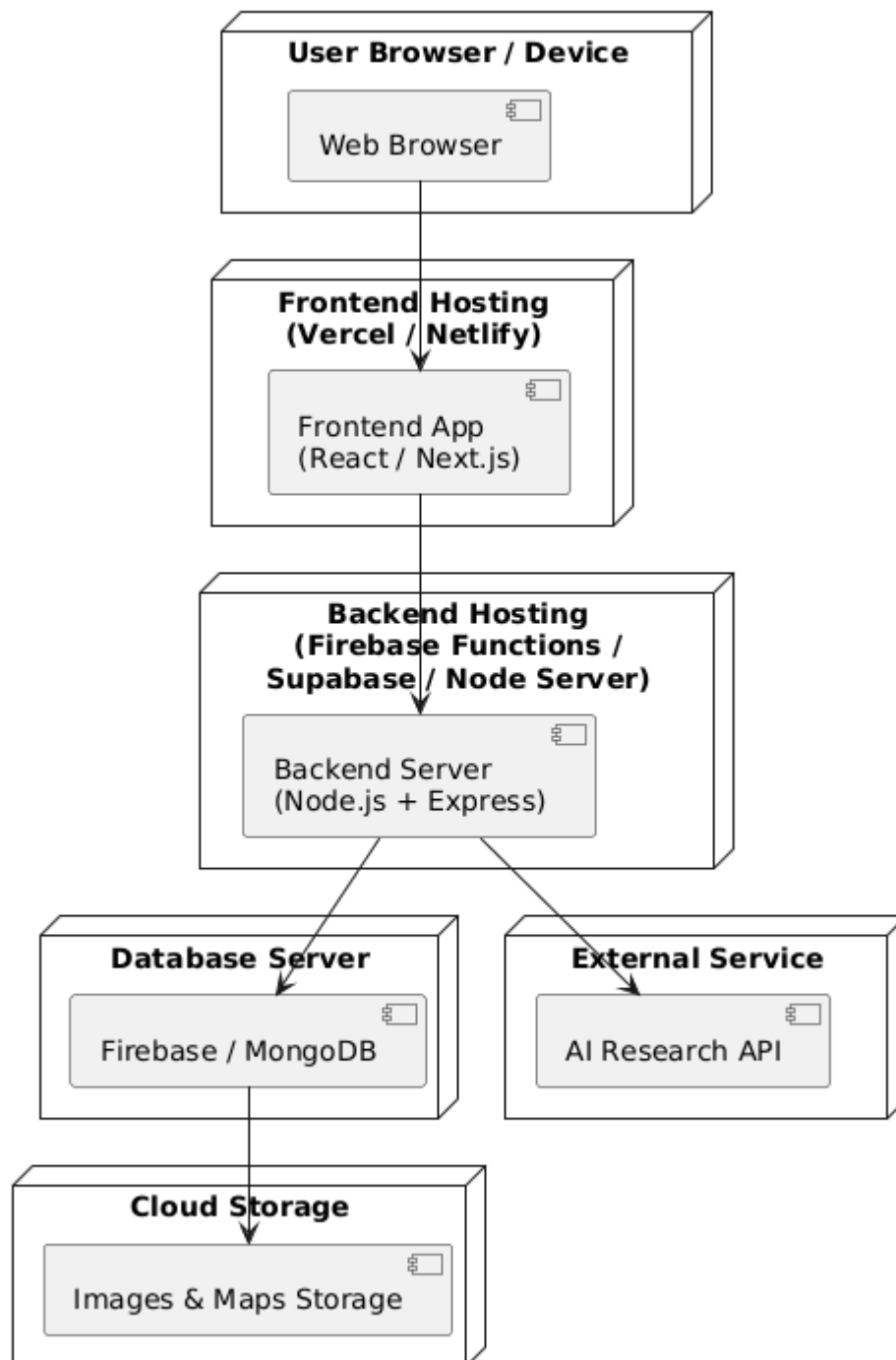
- **Devices:** A device is a node that is used to represent a physical computational resource in a system. An example of a device is an application server.

- **Deployment Specifications:** Deployment specifications is a configuration file, such as a text file or an XML document. It describes how an artifact is deployed on a node.

Deployment diagrams are used with the sole purpose of describing how software is deployed into the hardware system. It visualizes how software interacts with the hardware to execute the complete functionality. It is used to describe software to hardware interaction and viceversa.

DEPLOYMENT DIAGRAM FOR AUTHOR'S HAVEN:

Deployment Diagram - Author's Haven



1. Internet / Cloud

The users will be able to access the application via the internet using a web browser or mobile device.

2. Load Balancer (Traffic Distribution)

This will distribute the incoming traffic to multiple application servers.

This will ensure high availability and resource utilization efficiency.

3. Application Server Cluster

This will handle the application processing and will consist of:

Node.js – This will be the runtime environment for executing server-side JavaScript.

Express.js – This will be the backend framework that will handle APIs and business logic.

4. GraphQL API

This will be an efficient way to fetch and manipulate data.

This will enable structured communication between the frontend and backend.

5. Database Server (MyGraphDB)

This will store and manage the application data.

This will process and respond to GraphQL queries.

6. Frontend and Client-Side

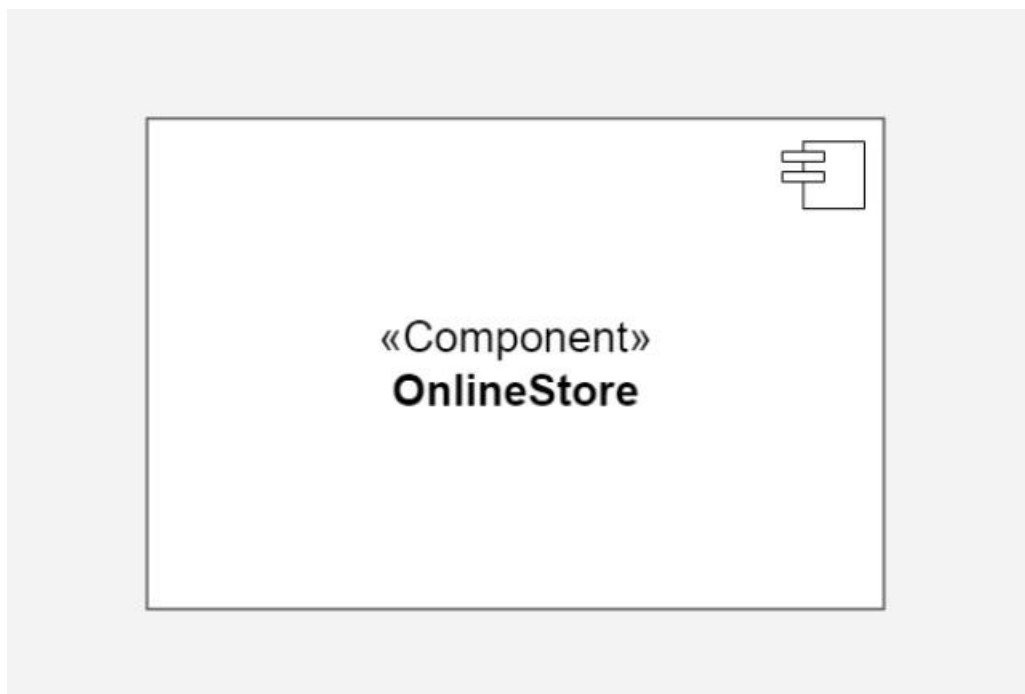
React.js – This will be the web frontend framework for building the user interface.

React Native App – This will be the mobile client for accessing the application on smartphones.

7.6 COMPONENT DIAGRAM

A component diagram is a UML diagram that shows the overall structure of a system by breaking it down into modular and replaceable components. It shows how different parts of a system interact with each other through interfaces and dependencies, giving a very clear architectural perspective of an application.

Component diagrams are employed to graphically represent the logical structure of software components and their relationships. They help in understanding how the functionalities of a system are allocated, how components interact with each other, and how services are delivered and received in a system. Unlike other UML diagrams that concentrate on behavior or physical deployment, component diagrams concentrate on the modular structure of software.



To create a component diagram, the following elements and notations need to be understood:

Components:

A component is a modular unit of the system that encapsulates its internal functionality and can be developed or replaced independently. It is usually depicted as a rectangle with the component name and sometimes an icon inside.

Interfaces:

Interfaces describe how components interact with each other.

The Provided Interface (depicted as a circle) represents the services provided by a component.

The Required Interface (depicted as a semicircle) represents the services required by a component.

Ports:

Ports are small squares drawn on the boundary of a component. They serve as interaction points where components exchange information with external modules.

Subsystems:

A subsystem is a specialized type of component that combines related components to form a major functional block of the system.

Artifacts:

Artifacts are physical entities like executable files, libraries, configuration files, or documents that implement or support components.

Relationships:

Component diagrams employ relationships to describe interactions between components:

Dependency: Indicates that a component depends on another component to work.

Association: Describes a structural relationship between components.

Aggregation: Describes a weak relationship where components can be independent.

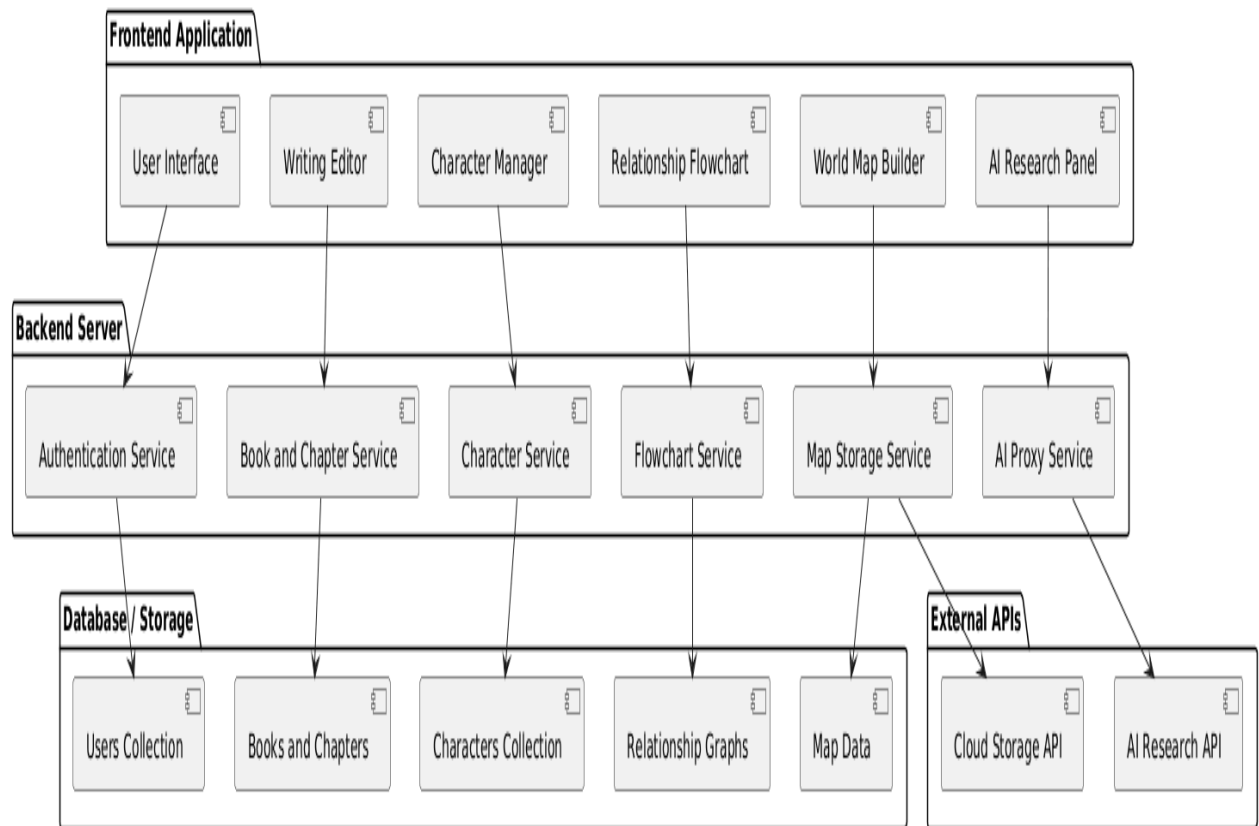
Composition: Describes a strong relationship where components depend on the whole.

Generalization: Describes inheritance between components.

Constraint: Defines rules or conditions applied to components.

COMPONENT DIAGRAM FOR AUTHOR'S HAVEN:

Component Diagram - Author's Haven



1. Frontend Application Components

User Interface

Writing Editor

Character Manager

Relationship Flowchart

World Map Builder

AI Research Panel

2. Backend Server Components

Authentication Service

Book and Chapter Service

Character Service

Flowchart Service

Map Storage Service

AI Proxy Service

3. Database / Storage Components

Users Collection

Books and Chapters

Characters Collection

Relationship Graphs

Map Data

4. External APIs

Cloud Storage API

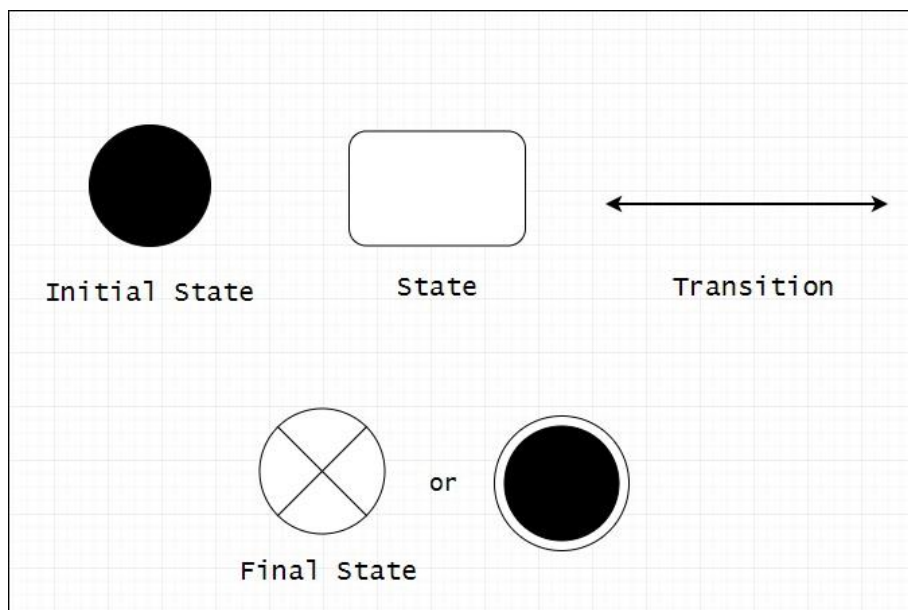
AI Research API

7.7 STATE CHART DIAGRAM :

State Machine Diagram is a UML behavioral diagram that is used to show the state or situation of a system at a particular point in time. It shows how a system reacts to events by moving from one state to another, thus modeling the dynamic behavior of a system.

State Machine diagrams are also referred to as State Diagrams or State-Chart Diagrams, and these terms are sometimes used interchangeably. They are mainly used to model how an object or a subsystem reacts to events that are either internal or external. By showing states and transitions, the diagram gives a clear indication of how the system changes over time.

This diagram is most useful when modeling systems that have more than one state and whose behavior is dependent on past states, such as user authentication systems, workflow systems, or interactive applications.



Basic Components and Notations of a State Machine Diagram

To design and analyze a State Machine Diagram, one needs to be aware of the following components:

Initial State:

The initial state is the starting point of the system or process. It is marked with a black-filled circle, which shows the initial point of the state machine's execution.

State:

The state is the condition or situation of an object at a particular point in time. It is represented by a rounded rectangle, which may contain internal actions or activities.

Transition:

The transition is the action of moving from one state to another. It is marked with a directed arrow labeled with the event or condition that causes the transition.

Fork:

The fork is used to represent a state divided into two or more parallel states. It is marked with a thick horizontal or vertical bar, which has one incoming transition and multiple outgoing transitions.

Join:

Join is used when there are several parallel states merging into one state. It is denoted by a thick line with several incoming transitions and one outgoing transition.

Self Transition:

Self transition is used when an event causes an action but does not alter the state. It is denoted by an arrow that turns back to the same state.

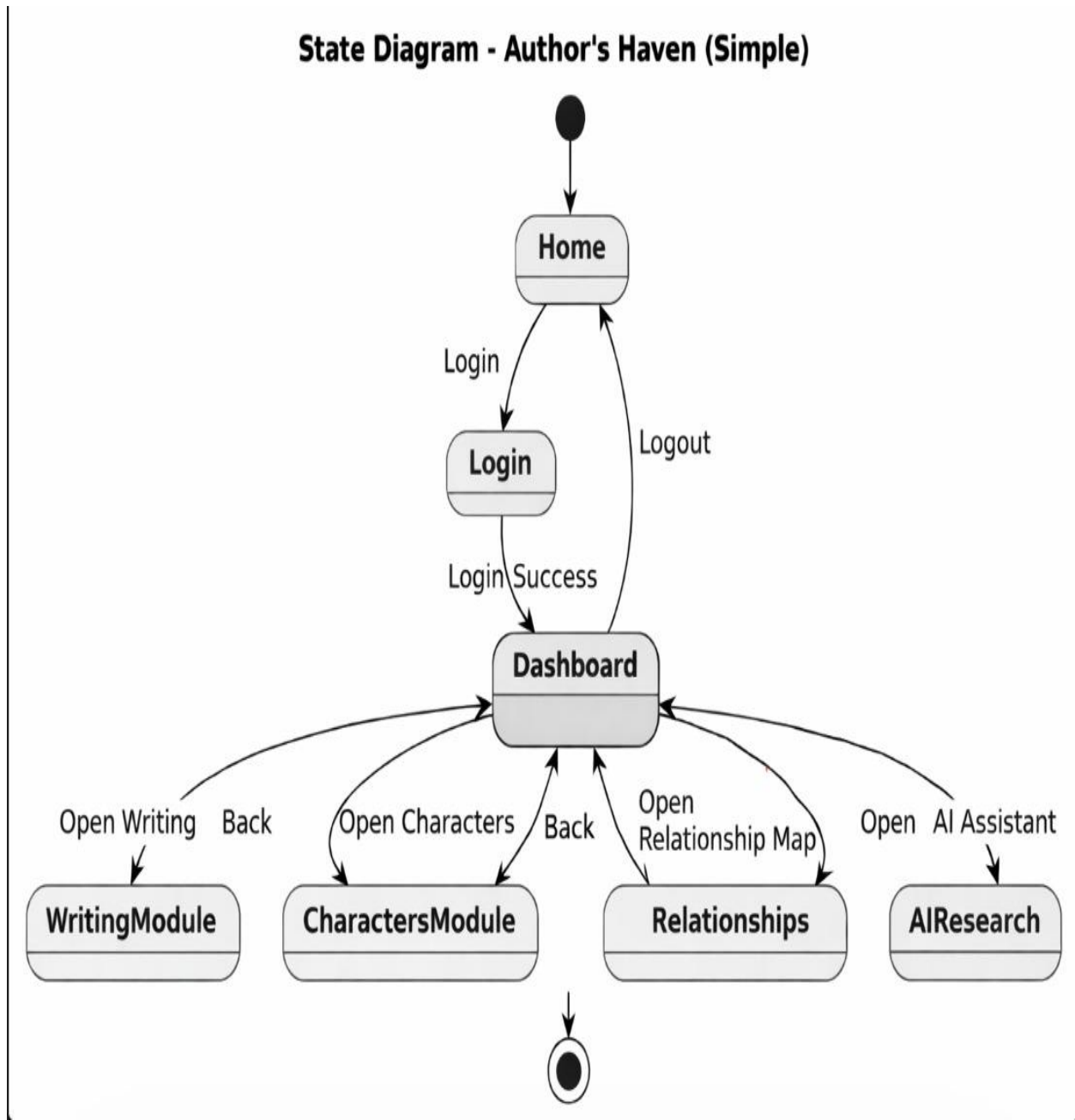
Composite State:

Composite state is a state that has other states nested inside it. It denotes complex behavior in a single state.

Final State:

The final state marks the end of the process. It is denoted by a filled circle inside another circle.

STATE CHART DIAGRAM FOR AUTHOR'S HAVEN:



1. Core Application Components

Home Component → Landing page and navigation entry point

Authentication Component → Handles login, session management, and logout

Dashboard Component → Central hub for accessing all modules

2. Feature Modules

Writing Module Component

Text editor

Chapter management

Autosave functionality

Characters Module Component

Character profile management

Character data storage and editing

Relationships Module Component

Relationship mapping

Visual connection management

AI Research Component

AI prompt handling

Research and suggestion generation

3. Supporting Components

Navigation Component → Handles transitions between dashboard and modules

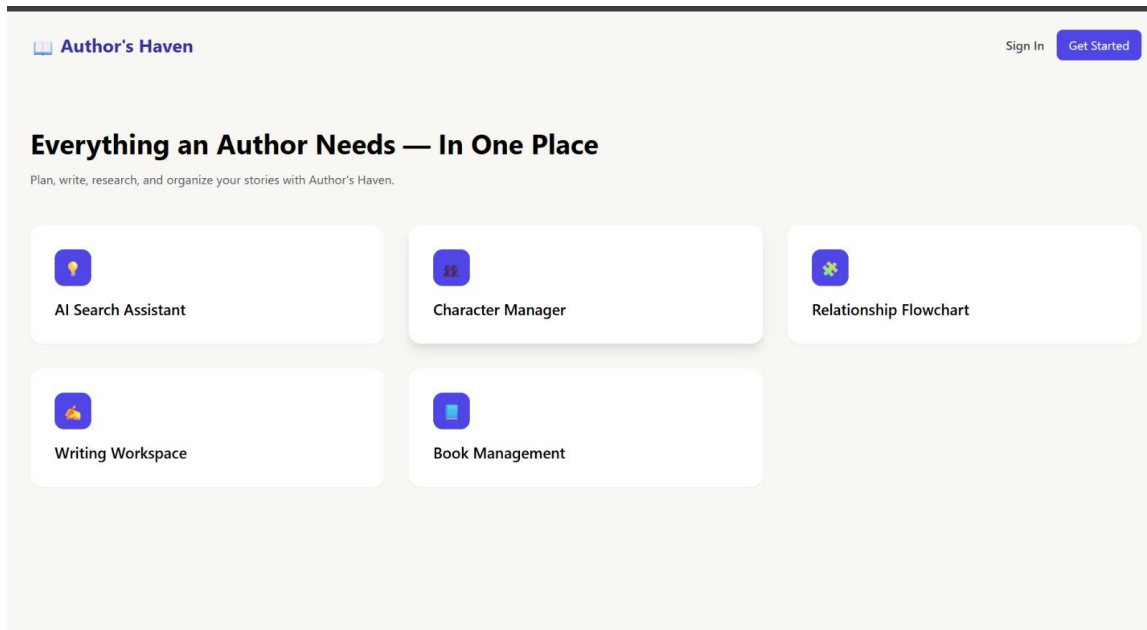
State Management Component → Maintains user session and UI state

API Service Component → Connects frontend modules to backend services.

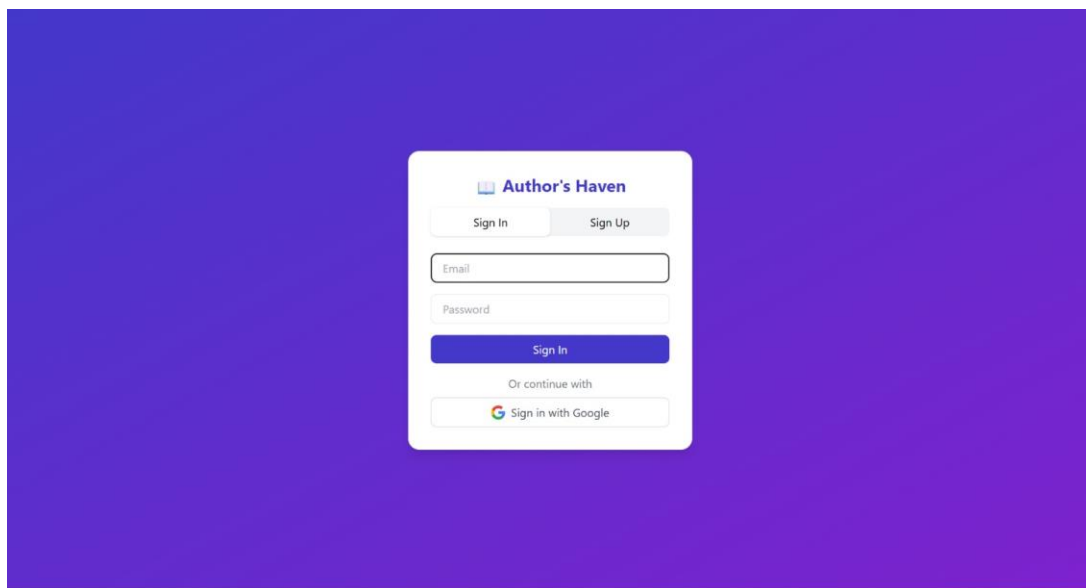
8. FRONTEND SCREENS

Models

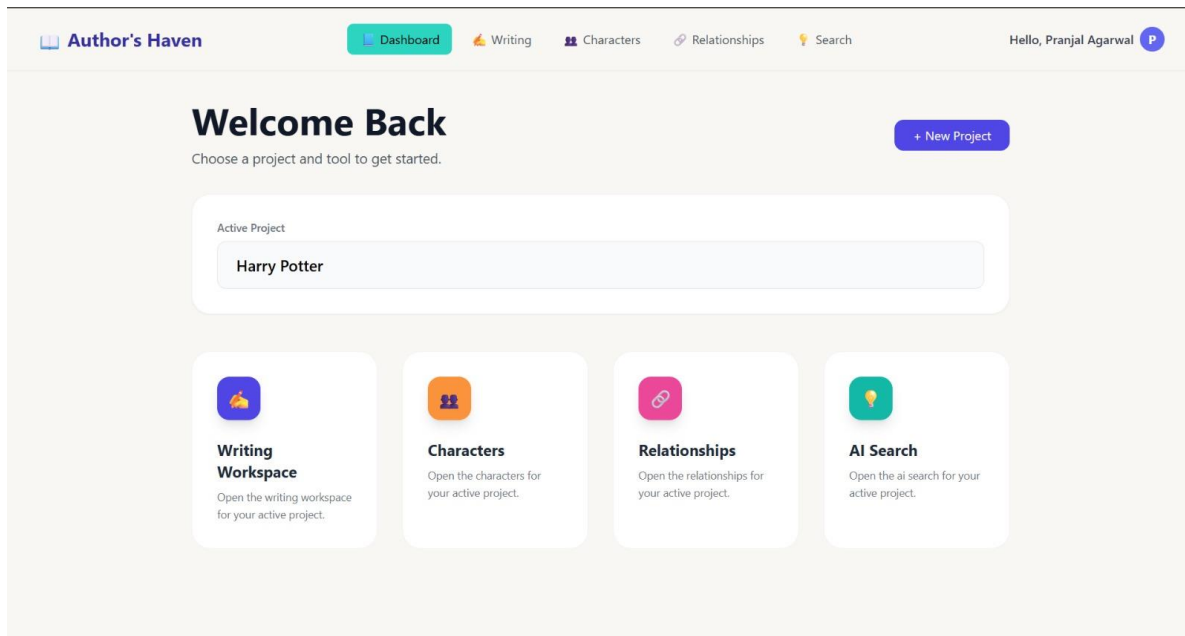
8.1 Main Page



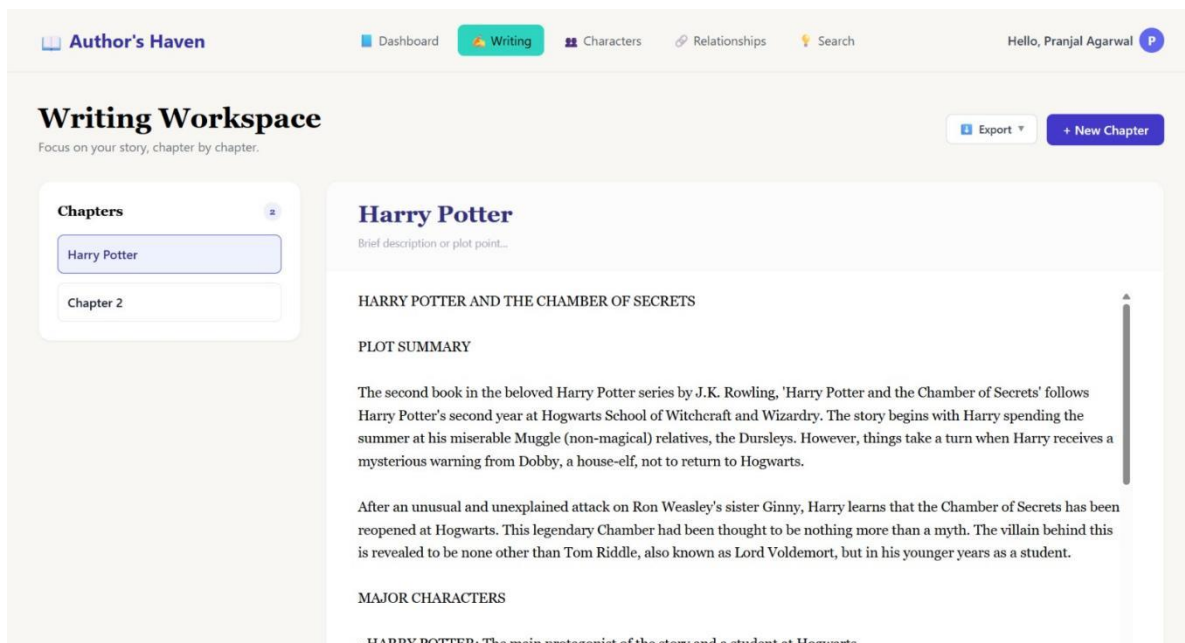
8.2 Login Page



8.3 Dashboard Page



8.4 Writing Page



8.5 Character Page

The screenshot shows the 'Character Manager' page in the 'Author's Haven' application. The top navigation bar includes 'Dashboard', 'Writing', 'Characters' (active), 'Relationships', and 'Search'. The user is logged in as 'Hello, Pranjal Agarwal'. The main heading is 'Character Manager' with an 'Export' button. Below this is a 'Create Character' form with fields for 'Character Name' (placeholder: 'Eg. Arya Stark') and 'Role' (placeholder: 'Hero, Villain, Sidekick...'). An 'Add Character' button is at the bottom of the form. Below the form are three character cards for 'harry', 'ron', and 'harmoine'. Each card has a 'Character' label, a description field (currently empty with the text 'No description provided yet.'), and 'Edit' and 'Delete' buttons.

8.6 Relationship Page

The screenshot shows the 'Character Relationships' page in the 'Author's Haven' application. The top navigation bar is the same as the previous page, but 'Relationships' is now the active tab. The main heading is 'Character Relationships' with a subtitle 'Define how your characters are connected in this project.' Below this are two main sections: '1. Quick Add Character' and '2. Define Relationship'. Section 1 has a 'Character name' input field and an 'Add' button. Section 2 has three dropdown menus: 'Character 1' (placeholder: 'Select'), 'Is a...' (placeholder: 'Friend'), and 'Of Character 2' (placeholder: 'Select'). An 'Add Relationship' button is at the bottom of section 2. Below these sections is a 'Connection Map' showing two relationships: 'FRIEND' between 'harry' and 'ron', and 'LOVE INTEREST' between 'ron' and 'harmoine'.

8.7 AI Search Page

AI Search Assistant

Get instant inspiration and research for your project.

Ask anything about your story or world... (Ctrl+Enter to submit)

Get AI Solution

A Latest Response

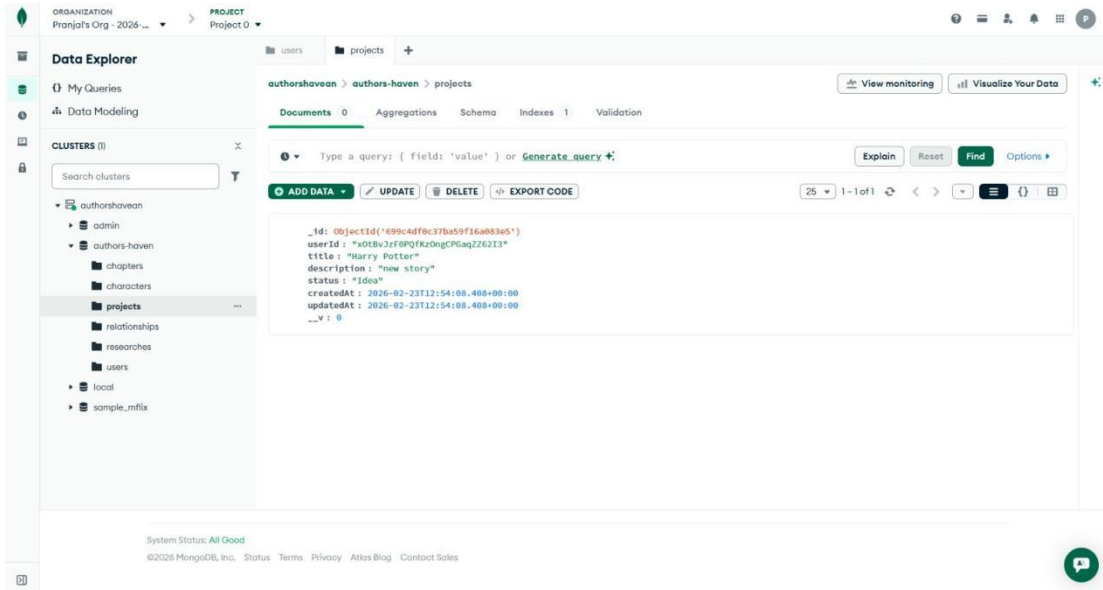
HARRY POTTER AND THE CHAMBER OF SECRETS

PLOT SUMMARY

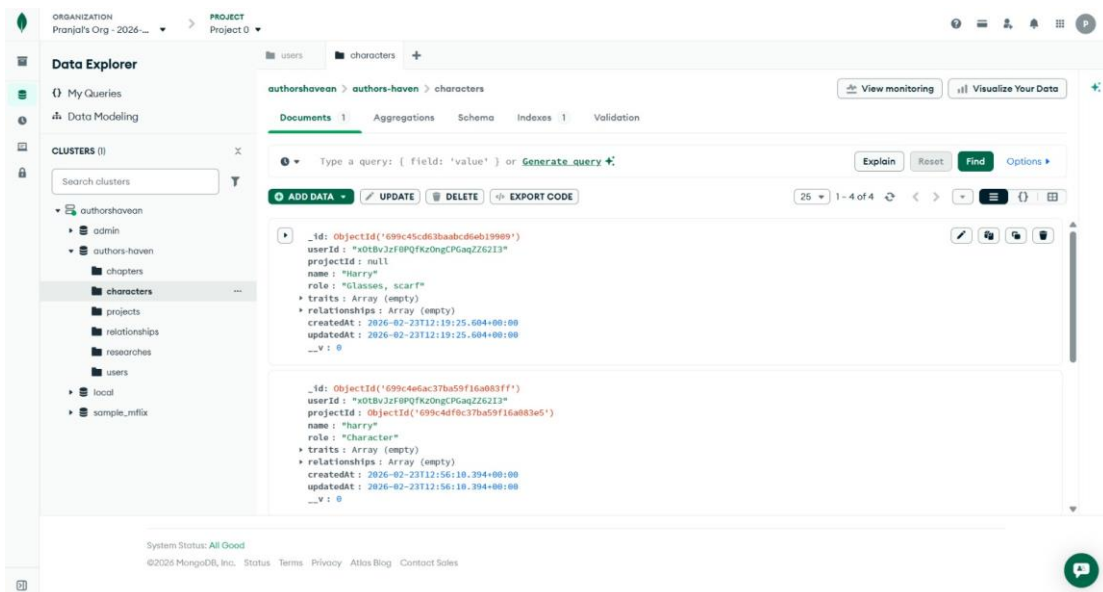
The second book in the beloved Harry Potter series by J.K. Rowling, 'Harry Potter and the Chamber of Secrets' follows Harry Potter's second year at Hogwarts School of Witchcraft and Wizardry. The story begins with Harry spending the summer at his miserable Muggle (non-magical) relatives, the Dursleys. However, things take a turn when Harry receives a mysterious warning from Dobby, a

9.BACKEND SCREENS

9.1 Project Data



9.2 Chapter Data



9.3 Character Data

The screenshot displays the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a tree view of clusters, with 'authors-haven' expanded and 'characters' selected. The main panel shows the 'characters' collection with two documents. The first document has fields: `_id`, `userId`, `projectId`, `name`, `role`, `traits`, `relationships`, `createdAt`, `updatedAt`, and `__v`. The second document is similar but with a different `projectId`. The interface includes a search bar, a query editor, and buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT CODE'. The bottom status bar indicates 'System Status: All Good' and '©2026 MongoDB, Inc.'.

9.4 Relationship Data

The screenshot displays the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a tree view of clusters, with 'authors-haven' expanded and 'relationships' selected. The main panel shows the 'relationships' collection with two documents. The first document has fields: `_id`, `projectId`, `from`, `to`, `relation`, and `__v`. The second document is similar but with a different `projectId`. The interface includes a search bar, a query editor, and buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT CODE'. The bottom status bar indicates 'System Status: All Good' and '©2026 MongoDB, Inc.'.

9.5 AI Data

The screenshot displays the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows a hierarchy of clusters and collections. The 'authorshavean' cluster is expanded, showing collections like 'admin', 'authors-haven', 'chapters', 'characters', 'projects', 'relationships', and 'researches'. The 'researches' collection is selected. The main panel shows the 'Documents' tab for the 'researches' collection. A query is entered in the search bar: 'Type a query: { field: 'value' } or [Generate query](#)'. Below the search bar, there are buttons for 'ADD DATA', 'UPDATE', 'DELETE', and 'EXPORT CODE'. The document preview shows a single document with the following fields: '_id', 'projectId', 'prompt', 'response', 'createdAt', 'updatedAt', and '_v'. The 'response' field contains the text: 'The Harry Potter series is a sequence of fantas...'. At the bottom, the system status is 'All Good' and the copyright notice is '©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales'.

ORGANIZATION: Pranjal's Org - 2026-... PROJECT: Project 0

Data Explorer

My Queries
Data Modeling

CLUSTERS (1)

Search clusters

- authorshavean
 - admin
 - authors-haven
 - chapters
 - characters
 - projects
 - relationships
 - researches
 - users
 - local
 - sample_mflix

authorshavean > authors-haven > researches

Documents 0 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT CODE

25 1-1 of 1

```
{
  "_id": ObjectId("699c4e1ec37ba59f16a883f7"),
  "projectId": ObjectId("699c4df0c37ba59f16a883e5"),
  "prompt": "harry potter ",
  "response": "HARRY POTTER OVERVIEW",
  "createdAt": 2026-02-23T12:54:54.068+00:00,
  "updatedAt": 2026-02-23T12:54:54.068+00:00,
  "_v": 0
}
```

System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

10.CODE

10.1 Homepage

```
1 import { useNavigate } from "react-router-dom";
2
3 export default function Home() {
4   const navigate = useNavigate();
5
6   const features = [
7     { title: "AI Search Assistant", icon: "💡" },
8     { title: "Character Manager", icon: "👤" },
9     { title: "Relationship Flowchart", icon: "🌿" },
10
11     { title: "Writing Workspace", icon: "🔥" },
12     { title: "Book Management", icon: "📖" },
13   ];
14
15   return (
16     <div className="min-h-screen bg-[#f8f7f3]">
17
18       {/* Header */}
19       <header className="flex justify-between items-center px-10 py-6">
20         <h1 className="text-2xl font-bold text-indigo-800">
21           Author's Haven
22         </h1>
23
24         <div className="flex gap-4">
25           <button
26             onClick={() => navigate("/auth")}
27             className="text-gray-700 font-medium hover:text-indigo-600"
28           >
29             Sign In
30           </button>
31
32           <button
33             onClick={() => navigate("/auth")}
34             className="bg-indigo-600 text-white px-4 py-2 rounded-lg hover:bg-indigo-700 transition"
35           >
36             Get Started
37           </button>
38         </div>
39       </header>
40
41       {/* Hero */}
42       <section className="px-10 py-16">
```

```

34     className="bg-indigo-600 text-white px-4 py-2 rounded-lg hover:bg-indigo-700 transition"
35   >
36     Get Started
37   </button>
38 </div>
39 </header>
40
41 /* Hero */
42 <section className="px-10 py-16">
43   <h2 className="text-4xl font-bold mb-4">
44     Everything an Author Needs – In One Place
45   </h2>
46
47   <p className="text-gray-600 max-w-2xl mb-12">
48     Plan, write, research, and organize your stories with Author's Haven.
49   </p>
50
51 /* Feature Cards */
52 <div className="grid grid-cols-1 md:grid-cols-3 gap-8">
53   {features.map((f, i) => (
54     <div
55       key={i}
56       onClick={() => navigate("/auth")}
57       className="bg-white p-8 rounded-2xl shadow-sm hover:shadow-lg transition cursor-pointer"
58     >
59       <div className="w-12 h-12 bg-indigo-600 text-white flex items-center justify-center rounded-xl text-xl mb-4">
60         {f.icon}
61       </div>
62
63       <h3 className="text-xl font-semibold">
64         {f.title}
65       </h3>
66     </div>
67   ))}
68 </div>
69 </section>
70 </div>
71 );
72 }

```

10.2 Chapterpage

```
1  import { useState } from "react";
2
3  export default function CreateChapter({ onCreate }) {
4    const [title, setTitle] = useState("");
5    const [description, setDescription] = useState("");
6
7    const handleSubmit = (e) => {
8      e.preventDefault();
9      if (!title.trim()) return;
10
11      if (typeof onCreate === "function") {
12        onCreate({
13          id: Date.now(),
14          title: title.trim(),
15          description: description.trim(),
16        });
17      }
18
19      setTitle("");
20      setDescription("");
21    };
22
23    return (
24      <div className="border rounded-lg p-4 bg-gray-50 mb-6">
25        <h3 className="font-serif font-semibold mb-3">New Chapter</h3>
26
27        <form onSubmit={handleSubmit} className="space-y-3">
28          <input
29            type="text"
30            placeholder="Chapter title"
31            value={title}
32            onChange={(e) => setTitle(e.target.value)}
33            className="w-full border rounded px-3 py-2"
34          />
35
36          <textarea
37            placeholder="Short description (optional)"
38            value={description}
39            onChange={(e) => setDescription(e.target.value)}
40            rows={3}
41            className="w-full border rounded px-3 py-2"
42          />
43        </form>
44      </div>
45    );
46  }
```

```

44     <button
45       type="submit"
46       className="w-full bg-indigo-700 text-white py-2 rounded"
47     >
48       Create Chapter
49     </button>
50   </form>
51 </div>
52 );
53 }

```

10.3 Characterpage

```

1 export default function CharacterList({ characters, onEdit, onDelete }) {
2   if (!characters || characters.length === 0) {
3     return (
4       <div className="text-center py-12 bg-white rounded-2xl shadow-sm border border-dashed border-gray-300">
5         <p className="text-gray-500 mb-4 text-lg">No characters created yet.</p>
6         <p className="text-indigo-500 font-medium">Use the form above to add your first character!</p>
7       </div>
8     );
9   }
10
11   return (
12     <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mt-8">
13       {characters.map((c) => (
14         <div
15           key={c._id || c.id}
16           className="bg-white p-6 rounded-xl shadow-sm hover:shadow-md transition-shadow border border-gray-100 flex flex-col j
17         >
18           <div>
19             <div className="flex justify-between items-start mb-3">
20               <h3 className="text-xl font-bold text-gray-800 truncate" title={c.name}>{c.name}</h3>
21               {c.role && (
22                 <span className="text-xs bg-indigo-50 text-indigo-700 px-2 py-1 rounded-full font-medium border border-indig
23                 {c.role}
24               </span>
25             )}
26           </div>
27
28           <p className="text-gray-600 mb-6 text-sm line-clamp-3">
29             {c.description || c.backstory || c.personality || "No description provided yet."}
30           </p>
31         </div>
32
33         <div className="flex justify-end gap-3 pt-4 border-t border-gray-50">
34           <button
35             onClick={() => onEdit && onEdit(c)}
36             className="text-indigo-600 hover:text-indigo-800 text-sm font-medium hover:underline px-2 py-1 rounded transition
37           >
38             Edit
39           </button>
40           <button
41             onClick={() => {

```

```

36     onClick={() => onEdit && onEdit(c)}
37     className="text-indigo-600 hover:text-indigo-800 text-sm font-medium hover:underline px-2 py-1 rounded transition"
38   >
39     Edit
40   </button>
41   <button
42     onClick={() => {
43       if (window.confirm("Are you sure you want to delete this character?")) {
44         onDelete && onDelete(c._id || c.id);
45       }
46     }}
47     className="text-red-500 hover:text-red-700 text-sm font-medium hover:underline px-2 py-1 rounded transition-color"
48   >
49     Delete
50   </button>
51 </div>
52 </div>
53 ))}
54 </div>
55 };
56 }

```

11.TESTING

A process of executing a program with the explicit intention of finding errors, that is making the program fail.

TESTING METHODOLOGIES:

- Black Box Testing
- Software Testing
- White Box Testing

SOFTWARE TESTING:

It is the process of testing the functionality and correctness of software by running it. A good test case is one that has a high probability of finding a yet undiscovered error.

Software Testing is usually performed for one of two reasons:

- Defect Detection
- Reliability Estimation

BLACK BOX TESTING:

Applies to software system or module, tests functionality in terms of inputs and outputs at interfaces. Test reveals if the software function is fully operational with reference to requirements specification.

WHITE/GLASS BOX TESTING:

Knowing the internal working i.e., to test if all internal operations are performed according to the program structure and data structure. To test if all internal components have been adequately exercised.

SOFTWARE TESTING STRATEGIES:

A Strategy for the software testing will begin in the following order:

- Unit Testing
- Integration Testing
- Validation Testing
- System Testing

UNIT TESTING:

It concentrates on each unit of the software as implemented in source code and is a white box oriented. Using the component level design description as a guide, important control paths are tested to uncover errors within the boundary of the module. Unit Testing Benefits:

- Unit testing increases confidence in changing/ maintaining code. If good unit tests are written and if they are run every time any code is changed.
- Able to promptly catch any defects introduced due to the change. In addition, if codes are already made less interdependent to make unit testing possible, the unintended impact of changes to any code is less.
- Codes are more reusable. In order to make unit testing possible, codes need to be modular. This means that codes are easier to reuse.

INTEGRATION TESTING:

Here focus is on design and construction of the software architecture. The objective is to take unit tested components and build a program structure that has been dictated by design.

Analogy

During the process of manufacturing a ballpoint pen, the cap, the body, the tail and clip, the ink cartridge and the ballpoint are produced separately and unit tested separately. When two or more units are ready, they are assembled and Integration Testing is performed. For example, whether the cap fits into the body or not.

VALIDATION TESTING:

Requirements established as part of software requirements analysis are validated against the software that has been constructed i.e., validation succeeds when software functions in a manner that can be reasonably expected by the customer.

Objective is to ensure that the product actually meets the user's needs and that the specifications were correct in the first place. In other words, to demonstrate that the product fulfils its intended use when placed in its intended environment.

SYSTEM TESTING:

In this software and other system elements are tested as a whole.

Analogy

During the process of manufacturing a ballpoint pen, the cap, the body, the tail, the ink cartridge and the ballpoint are produced separately and unit tested separately. When two or more units are ready, they are assembled and Integration Testing is performed. When the complete pen is integrated, System Testing is performed.

12. IMPLEMENTATION

Implementation is the process of executing and deploying the designed system into a real-time working environment. To implement Author's Haven, the following operations are performed:

The platform is compatible with laptops and desktop computers using a modern web browser.

Open an updated browser like Chrome, Edge, or Firefox.

Type in the deployed URL of Author's Haven in the address bar.

The homepage will load, allowing users to log in or sign up.

Login using registered credentials like Email and Password or Google Sign-In.

After successful authentication, the user dashboard will appear.

Users can create, edit, delete, and export books.

Writers can manage chapters by adding, editing, deleting, and rearranging them with autosave.

Users can create and manage character profiles and establish relationships between characters.

The AI Assistant allows users to generate story ideas, enhance writing, and research topics.

All data is safely stored in the MongoDB database via the backend API.

The system provides safe access with JWT authentication and protected routes.

13. MAINTAINENCE

It is feasible to create a system that satisfies the current requirements of the users; however, the needs of the users may change over time as the usage of the platform increases. Author's Haven will need to be maintained periodically to accommodate the changing requirements and ensure continued efficiency and effectiveness.

Software systems require modifications after their implementation due to various reasons, including user feedback, performance enhancement, bug correction, and technological advancements. Any modification or update may affect the functionality of the system, and hence, flexible and scalable approaches to maintenance should be adopted.

With the advancement of web technologies, Author's Haven can be developed to incorporate new functionalities, better performance, and improved user experience. Since the platform uses a modular MERN stack architecture, new developments can be easily incorporated without interfering with the existing functionalities.

Security is an important consideration for system maintenance. With the advancement of cybersecurity threats, security features such as JWT authentication, handling APIs, and database security approaches will be updated periodically to protect the users' data and ensure system integrity.

14.CONCLUSION

In summary, Author's Haven is an innovative solution for digital writing and world-building that incorporates contemporary full-stack technology into a single creative platform. Author's Haven is developed using MERN stack architecture and AI technology, ensuring a scalable, secure, and friendly interface for writers.

Using Author's Haven's friendly interface, easy navigation, and organized project management, writers can easily organize their books, chapters, characters, and relationships. The AI Assistant feature of Author's Haven boosts creativity by developing story ideas, enhancing writing quality, and assisting with research in the same environment.

Author's Haven also provides secure login access using JWT authentication and organized database management using MongoDB. The system's modular architecture enables effortless communication between the frontend and backend, ensuring efficient data management and optimization.

As the field of creative writing continues to incorporate digital technology, Author's Haven is a complete and intelligent solution that makes writing easier while still being organized and secure. With its scalable architecture and AI technology, Author's Haven is poised for future development and innovation in the realm of digital writing and world-building.

15. BIBLIOGRAPHY

Sites-Referred:

- <https://www.geeksforgeeks.org/>
- <https://app.hygraph.com/>
- <https://www.javatpoint.com/>

<https://www.w3schools.com/java/>

<https://www.techopedia.com/definition/23889/webdevelopment>

